
AI-POWERED HEALTH REMINDER SYSTEM AND EMERGENCY ALERT APP

A PROJECT REPORT (PHASE-2)

by

FEBA M AJI (VJC22CSD026)

SHIBANA NAZEER (VJC22CSD059)

SREELAKSHMI SAI (VJC22CSD061)

JYOTHYLAKSHMI BIJU (VJC22CSD037)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND DESIGN

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY



**DEPARTMENT OF COMPUTER SCIENCE AND DESIGN
VISWAJYOTHI COLLEGE OF ENGINEERING AND
TECHNOLOGY, VAZHAKULAM
MARCH 2026**

AI-POWERED HEALTH REMINDER SYSTEM AND EMERGENCY ALERT APP

A PROJECT REPORT (PHASE-2)

by

FEBA M AJI (VJC22CSD026)

SHIBANA NAZEER (VJC22CSD059)

SREELAKSHMI SAI (VJC22CSD061)

JYOTHYLAKSHMI BIJU (VJC22CSD037)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND DESIGN

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

under the guidance

of

Mrs. PRETTY SEBASTIAN

Assistant Professor, CG Dept.



DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

VISWAJYOTHI COLLEGE OF ENGINEERING AND

TECHNOLOGY, VAZHAKULAM

MARCH 2026

VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

Department of Computer Science and Design

Vision

Moulding socially dynamic Computer Science and Design engineers capable of driving technological advancements

Mission

1. To provide comprehensive education that combines technical knowledge, creativity and critical thinking through computer science principles and design methodologies.
2. Promote research and industry collaboration to empower our graduates for addressing real world challenges.
3. To cultivate ethical values, social responsibility and a commitment to lifelong learning among students.

Program Educational Objectives

Our Graduates

1. Shall have strong foundation in theoretical and practical aspects of Computer Science and Design principles to develop innovative solutions.
2. Shall possess effective communication, teamwork and leadership skills to collaborate with diverse stakeholders and to practice in a corporate firm.
3. Shall have potential to become entrepreneurs, innovators and pursue research in Computer Science and Design or related fields.
4. Shall have a strong sense of social awareness and stay updated with evolving design technologies.

Program Outcomes

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences

3. **Design / development of solutions:**Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety and the cultural, societal and environmental considerations.
4. **Conduct investigations of complex problems:**Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:**Create, select and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:**Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:**Understand the impact of the professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of and need for sustainable development.
8. **Ethics:**Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice
9. **Individual and team work:**Function effectively as an individual and as a member or leader in diverse teams and in multidisciplinary settings
10. **Communication:**Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and unread in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:**Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes

1. Ability to integrate theory and practice to design software applications.
2. Able to apply computer science skills and design tools to analyse, design and model composite systems.
3. Ability to design and manage projects to develop a career in a related industry.

**VISWAJYOTHI COLLEGE OF ENGINEERING AND
TECHNOLOGY, VAZHAKULAM**
DEPARTMENT OF COMPUTER SCIENCE AND DESIGN



BONAFIDE CERTIFICATE

This is to certify that the project report (phase-2) entitled “ **AI-Powered Health Reminder System and Emergency Alert App** ” is a bonafide record of the work done by **SREELAKSHMI SAI (VJC22CSD061)**, **JYOTHYLAKSHMI BIJU (VJC22CSD037)**, **SHIBANA NAZEER (VJC22CSD059)**, **FEBA M AJI (VJC22CSD026)** in partial fulfillment of the requirements for the award of the **Degree of Bachelor of Technology in Computer Science and Design** of APJ Abdul Kalam Technological University.

Internal Supervisor

External Supervisor

Project Coordinator

Head of Department

ACKNOWLEDGEMENT

First and foremost, we thank **God Almighty** for His divine grace and blessings in making all these possible. May He continue to lead us in the years. It is our privilege to render our heartfelt thanks to our beloved Manager, **Rev. Msgr. Dr. Pius Malekandathil**, our Director, **Rev. Fr. Paul Parathazham**, and our Principal **Dr. K K Rajan** for providing us the opportunity to do this project report (phase-2) during the Fourth year (2026) of my B.Tech degree course. We are deeply thankful to our Head of the Department, **Mrs. Sabitha Raju** for her support and encouragement. We would like to express our sincere gratitude and heartfelt thanks to our Project Guide, **Mrs. Pretty Sebastian**, Assistant Professor, Department of Computer Science and Design, for her motivation, assistance, and help for the project. We also express our sincere thanks to our Project Coordinator, **Mrs. Anju Markose**, Assistant Professor, Department of Computer Science and Design, for her guidance and support. We also thank all the staff members of the Computer Science and Design department for providing their assistance and support. Last, but not least, we would like to thank all our friends and family for their valuable feedback from time to time, as well as their help and encouragement.

DECLARATION

We undersigned hereby declare that the project report “AI-POWERED HEALTH REMINDER SYSTEM AND EMERGENCY ALERT APP”, submitted for partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology of the APJ Abdul Kalam Technological University is a bonafide work done by ourself under the supervision of Mrs. Pretty Sebastian. This submission represents ideas in our own words, and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or formed the basis for the award of any degree, diploma, or similar title of any other University.

Place: Vazhakulam

Date: 31-03-2026

FEBA M AJI

SHIBANA NAZEER

SREELAKSHMI SAI

JYOTHYLAKSHMI BIJU

ABSTRACT

Managing medication schedules and monitoring the health of elderly and chronically ill patients can be challenging for both patients and caregivers, as missed medicines or unnoticed emergencies may lead to serious health complications. This project presents an **AI-Powered Health Reminder System and Emergency Alert Application**, developed as a mobile-based solution to assist patients in managing their daily medication and healthcare activities. The application provides timely reminders for medicines and allows patients to log their medication intake, while automatically notifying caregivers if a scheduled dose is missed. The system also includes features such as doctor appointment reminders, emergency alert notifications, voice navigation for easy interaction, and a caregiver dashboard that enables remote monitoring of the patient's medication history and activity status. In emergency situations or abnormal inactivity, the system sends immediate alerts to the caregiver so that appropriate actions can be taken. By integrating intelligent reminders, monitoring mechanisms, and user-friendly features, the proposed system aims to improve medication adherence, enhance patient safety, and strengthen communication between patients and caregivers, thereby supporting elderly individuals in maintaining their independence while ensuring continuous care and supervision.

Contents

List of Figures	i
List of Tables	iii
List of Abbreviations	iii
1 INTRODUCTION	1
1.1 Problem Statement	2
1.2 Objective	2
1.3 Scope	3
2 LITERATURE SURVEY	4
2.1 Intelligent and Safe Medication Box in Health IoT Platform for Medication Monitoring System with Timely Reminders	4
2.1.1 Methodology of Intelligent Medication Monitoring System	4
2.1.2 Model Architecture	5
2.1.3 Advantages	6

2.1.4	Disadvantages	6
2.2	MEMO Box: Health Assistant for Depression With Medicine Carrier and Exercise Adjustment	7
2.2.1	Methodology	7
2.2.2	Model Architecture	8
2.2.3	Advantages	9
2.2.4	Disadvantages	9
2.3	IoT-Based Medical Assistant for Efficient Monitoring of Patients in Response to COVID-19	10
2.3.1	Methodology of IoT-Based Patient Monitoring System	10
2.3.2	Model Architecture	11
2.3.3	Advantages	11
2.3.4	Disadvantages	12
2.4	ACTIVA: Innovation in Quality of Care for Nursing Homes Through Activity Recognition	13
2.4.1	Methodology	13
2.4.2	Model Architecture	13
2.4.3	Advantages	14
2.4.4	Disadvantages	14
2.5	Voice Assistant for Health Self-Management for Older Adults	15

2.5.1	Methodology	15
2.5.2	Model Architecture	16
2.5.3	Advantages	17
2.5.4	Disadvantages	17
3	PROPOSED SYSTEM	18
3.1	Process Overview	19
3.1.1	Process Flow Diagram	21
3.1.2	Architecture Diagram	22
3.1.3	Use case Diagram	22
3.1.4	Data flow diagram	23
3.1.5	Data flow diagram	23
3.1.6	Data flow diagram	24
3.1.7	Class Diagram	24
3.2	System Requirements	24
3.2.1	Hardware Requirements	24
3.2.2	Software Requirements	25
3.2.3	Flutter	25
3.2.4	MySQL	25

3.2.5 Python	25
3.3 Implementation Details	26
3.3.1 System Architecture	26
3.3.2 Mobile Application Implementation	26
3.3.3 Caregiver Monitoring Module	26
3.3.4 Backend Implementation	27
3.3.5 Database Implementation	27
3.3.6 Notification and Alert System	27
3.3.7 Security Implementation	28
3.3.8 Testing and Validation	28
3.4 Methodology	28
3.4.1 Project Planning and Requirement Analysis	28
3.4.2 System Design	29
3.4.3 System Implementation	29
3.4.4 Functional Module Development	29
3.4.5 Testing and Quality Assurance	30
3.4.6 Deployment and Maintenance	30

Appendix A	SERVER	iii
Appendix B	User Interface	xx
Appendix C	Screenshots	1

List of Figures

2.1	Architecture of Intelligent Medication Box System	5
2.2	Architecture of MEMO Box Health Assistant System	8
2.3	Architecture of IoT Based Patient Monitoring System	11
2.4	ACTIVA system architecture based on a fog-cloud architecture	14
2.5	Architecture of Voice Assistant Health Self-Management System	16
3.1	Process Flow Diagram	21
3.2	Architecture Diagram	22
3.3	Use case diagram	22
3.4	Data Flow Diagram Level 0	23
3.5	Data Flow Diagram Level 1-User	23
3.6	Data Flow Diagram Level 2-User	24
3.7	Class diagram	24
C.1	Sign-In Page	1
C.2	Admin Page	1
C.3	Patient Management	li

C.4 Registration	li
C.5 Caregiver Registration	lii
C.6 Doctor Appointment Schedule	lii
C.7 Medicine Details	liii
C.8 Missed Medicine	liii
C.9 Emergency Contact	liv
C.10Emergency Calls	liv
C.11Contact Person	lv
C.12Doctor Appointments	lv
C.13Upcoming Appointments	lvi
C.14Voice Command	lvi
C.15Patient's Home Page	lvii

List of Abbreviations

AI	Artificial Intelligence
ML	Machine Learning
API	Application Programming Interface
GPS	Global Positioning System
SMS	Short Message Service
UI	User Interface
UX	User Experience
DB	Database
SQL	Structured Query Language
OS	Operating System
RAM	Random Access Memory
LLM	Large Language Model

Chapter 1

INTRODUCTION

Healthcare management has become increasingly important, especially for elderly individuals and patients suffering from chronic illnesses who must follow strict medication schedules and attend regular medical appointments. Forgetting to take medicines on time or missing medical check-ups can lead to serious health complications and may worsen existing medical conditions. In many cases, caregivers or family members are responsible for monitoring the patient's health and ensuring that medicines are taken regularly. However, continuous monitoring is often difficult due to busy lifestyles, distance, or lack of proper tools that provide real-time updates about the patient's condition.

Traditional reminder systems mainly rely on simple alarms or notifications that remind users to take medicines at specific times. While these systems may help patients remember their medication, they do not provide intelligent monitoring, medication tracking, or emergency support. Additionally, many existing applications are not designed with elderly users in mind, making them difficult to use due to complex interfaces and limited accessibility features. Therefore, there is a need for a more advanced and user-friendly healthcare assistance system.

To address these challenges, this project proposes an AI-Powered Health Reminder System and Emergency Alert Application. The system is designed as a mobile application that provides intelligent medicine reminders, appointment notifications, and emergency alerts to improve patient safety and healthcare management. Patients receive reminders to take their medicines at scheduled times and can log their medication intake through the application. If the patient fails to take the medicine within the specified time, the system automatically sends a notification to the caregiver.

The application also includes additional features such as voice navigation, which allows elderly users to interact with the system using voice commands, making the application easier to operate. In emergency situations or abnormal inactivity, the system sends immediate alerts to the caregiver so that appropriate actions can be taken. The caregiver dashboard enables caregivers to monitor patient activities, medication history, missed doses, and emergency alerts in real time.

The application also includes additional features such as voice navigation, which allows elderly users to interact with the system using voice commands, making the application easier to operate. In emergency situations or abnormal inactivity, the system sends immediate alerts to the caregiver so that appropriate actions can be taken. The caregiver dashboard enables caregivers to monitor patient activities, medication history, missed doses, and emergency alerts in real time.

1.1 Problem Statement

Managing medication schedules and healthcare routines is a major challenge for elderly individuals and patients with chronic illnesses who must take medicines at specific times and attend regular medical appointments. Patients may forget to take their medicines due to memory issues, busy lifestyles, or a lack of proper reminders, which can lead to serious health complications. Traditional reminder systems usually provide simple alarms but do not confirm whether the medicine has actually been taken, and they do not offer real-time monitoring or emergency support. In many cases, caregivers or family members cannot continuously monitor the patient, especially when they are not physically present, which increases the risk of missed medications and unnoticed emergencies. Therefore, there is a need for a smart and reliable system that can remind patients to take medicines on time, track medication intake, notify caregivers if medicines are missed, and send alerts during emergency situations while remaining simple and accessible for elderly users.

1.2 Objective

1. Develop a user-friendly mobile application that provides intelligent medicine reminders for elderly and chronically ill patients.
2. Ensure proper medication adherence by tracking medicine intake and notifying caregivers if doses are missed.
3. Detect emergency situations using device sensors and send alerts to caregivers for timely assistance.
4. Provide a caregiver dashboard for monitoring patient activities, medication status, and health updates.
5. Support additional features such as voice navigation, appointment reminders, and medicine stock monitoring to improve healthcare management.

1.3 Scope

1. Users can set up personalized medicine schedules, and the system will automatically provide intelligent reminders through notifications and voice alerts at the specified times.
2. The application can detect user inactivity or sudden movements using mobile sensors such as the accelerometer and gyroscope, helping identify potential emergencies like falls or long periods of immobility.
3. In case of emergencies, the system can automatically send instant SMS alerts to registered caregivers using the Twilio API, ensuring timely assistance and safety.
4. The app includes a facial recognition feature that allows authorized users to cancel false alerts through face verification, improving reliability and reducing unnecessary panic.
5. Caregivers can monitor users' health activities, medicine logs, and emergency status in real time through the connected dashboard powered by Firebase Cloud integration.
6. The system supports voice commands and multilingual interaction to make the application user-friendly and accessible for elderly individuals with limited technical knowledge.
7. Future enhancements may include integration with wearable devices and IoT-based health sensors for advanced real-time monitoring and predictive health alerts.
8. The application securely stores user data, medicine history, and alert logs in a cloud database, ensuring data privacy, reliability, and easy access for future reference.
9. The system can operate efficiently in both online and offline modes, allowing users to receive scheduled reminders even without an active internet connection.
10. The app provides an intuitive user interface with large icons, clear navigation, and high-contrast visuals designed specifically to enhance usability for elderly and visually impaired users.

Chapter 2

LITERATURE SURVEY

2.1 Intelligent and Safe Medication Box in Health IoT Platform for Medication Monitoring System with Timely Reminders

The rapid development of Internet of Things (IoT) technology has significantly improved health-care services, especially for elderly people and patients suffering from chronic diseases. These patients need to take medicines regularly at the correct time and dosage. However, due to busy schedules or memory issues, patients and caregivers may forget medication schedules, which can lead to serious health problems. To address this issue, an intelligent medication box integrated with an IoT platform has been proposed to monitor medicine intake and provide timely reminders.

The proposed system contains six sub-compartments to organize different medicines and uses an Android application to send reminders to patients or caregivers. The system also integrates biosensors to monitor health parameters such as temperature and heartbeat. An authentication mechanism ensures that only authorized users can access the medication box, preventing misuse or overdosage. The system also tracks medicine stock and provides alerts when refilling is required. This intelligent medication box improves medication adherence and ensures safe and effective healthcare monitoring[1].

2.1.1 Methodology of Intelligent Medication Monitoring System

The proposed system is based on an IoT-enabled medication monitoring framework. The system uses an Arduino microcontroller that manages sensors, pill dispensing mechanisms, and communication with the mobile application. Biosensors are used to monitor the patient's health conditions, including temperature and heart rate.

The medication box includes six compartments that store different medicines. When the scheduled time arrives, the system sends alerts through LEDs, sound signals, and notifications to the Android

application. The internal timer controls the dispensing of pills at specific intervals. The caregiver initially enters medicine details such as dosage, quantity, and schedule through the mobile application. The microcontroller processes this information and automatically releases the correct pill at the scheduled time.

2.1.2 Model Architecture

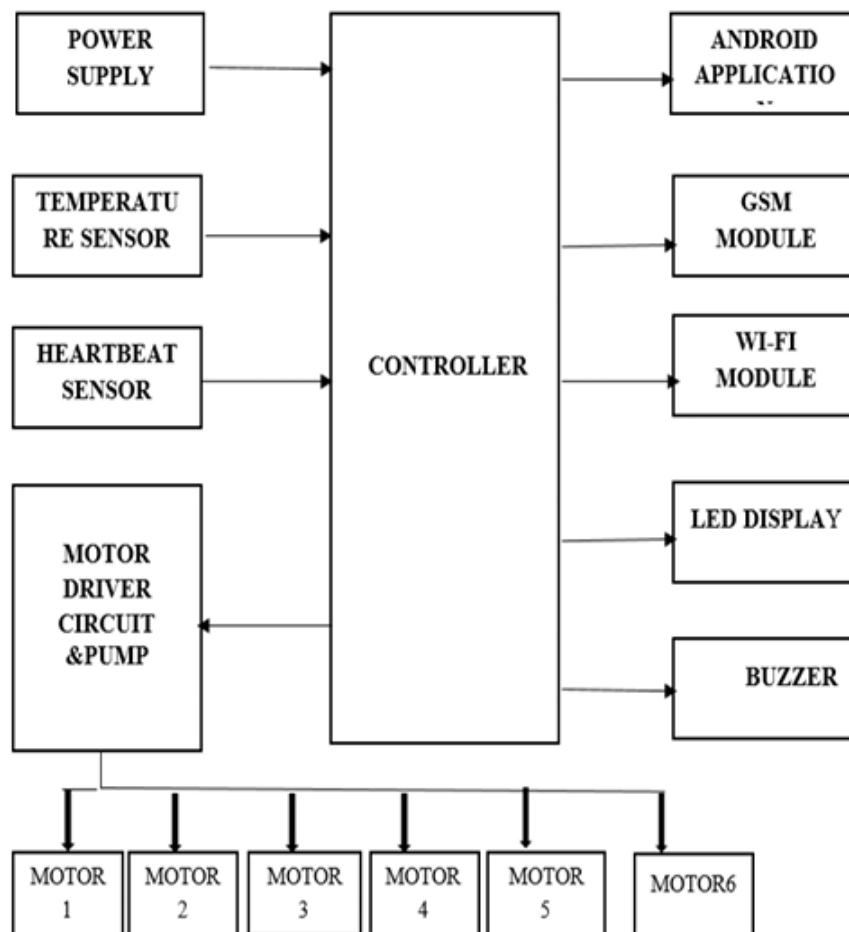


Figure 2.1: Architecture of Intelligent Medication Box System

The architecture of the system consists of several hardware and software components working together to ensure proper medication monitoring. The main component of the system is the Arduino microcontroller, which acts as the central processing unit of the medication box. It receives data from various sensors and controls the overall operation of the system.

The biosensors, such as temperature and heartbeat sensors, are used to collect important health information from the patient. This data helps in monitoring the patient's health condition and can be used for further medical analysis if required. The collected data is sent to the microcontroller for processing.

The pill storage unit consists of multiple compartments where different medicines are stored separately. These compartments are connected to a motor controller that operates the pill dispensing

mechanism. Based on the scheduled time set in the system, the microcontroller sends signals to the motor controller, which activates the stepper motors to release the required pill into the tray.

The system also includes a Wi-Fi module that connects the medication box to the Internet of Things (IoT) platform. Through this connection, the system communicates with an Android mobile application where caregivers or patients can set medicine schedules, dosage information, and refill alerts.

Additionally, the notification system consists of LEDs and buzzer alerts that remind the patient to take the medicine at the correct time. Notifications are also sent to the smartphone application to ensure that the patient or caregiver receives timely reminders. This integrated architecture ensures accurate medication management, remote monitoring, and improved healthcare support for elderly patients.

2.1.3 Advantages

1. **Convenient Medication Management:** The system helps elderly patients remember their medication schedule through automated reminders and alerts.
2. **Health Monitoring Capability:** Integrated biosensors monitor important health parameters such as body temperature and heartbeat.
3. **Improved Safety:** The authentication process prevents unauthorized access and reduces the risk of medication misuse.
4. **Remote Monitoring:** Caregivers can monitor patient medication status through the connected mobile application.
5. **Health Monitoring Support:** The integration of biosensors allows monitoring of important health parameters such as body temperature and heartbeat, which helps in better healthcare supervision.

2.1.4 Disadvantages

1. **Hardware Complexity:** The integration of sensors, motors, and controllers increases the complexity of the hardware system.
2. **Dependency on Internet Connectivity:** The system requires stable internet connectivity for communication with the mobile application.
3. **Maintenance Requirement:** Sensors and electronic components require regular maintenance for proper functioning.

2.2 MEMO Box: Health Assistant for Depression With Medicine Carrier and Exercise Adjustment

Depression is a common mental health problem that affects many people worldwide. Patients suffering from depression often forget to take their medications on time and may lack motivation to follow healthy routines such as exercise. To address this problem, an intelligent health assistant system called MEMO Box is proposed. The system helps patients manage their medication schedules while also encouraging physical activity. It integrates a medicine carrier with a smart monitoring system that can remind patients to take medicines and perform recommended exercises. The system aims to support mental health treatment by improving medication adherence and promoting a healthier daily routine.

The MEMO Box system is designed to provide continuous assistance to patients by combining medication management with daily activity support. It uses intelligent reminder mechanisms to notify users about their medication schedules and recommended exercise routines. By monitoring the user's interaction with the device, the system can help ensure that medicines are taken at the correct time. This integrated approach not only supports treatment adherence but also promotes healthier lifestyle habits, which are important for improving the overall well-being of individuals suffering from depression[2].

2.2.1 Methodology

The MEMO Box system is designed to monitor medication intake and assist patients suffering from depression. The system uses a microcontroller-based hardware unit connected with sensors, a medicine storage compartment, and communication modules. The device provides reminders for medicine intake and suggests exercise activities to improve the patient's mental health condition. Data related to medicine schedules and user activities can be monitored through a connected application or interface. The system automatically notifies the patient when it is time to take medication and encourages physical activity through scheduled reminders. By combining medication management and exercise recommendations, the system provides a supportive tool for patients undergoing depression treatment.

In addition, the MEMO Box system incorporates a scheduling mechanism that stores medication timings and exercise routines in the microcontroller memory. When the scheduled time arrives, the system activates reminder alerts such as alarms, LEDs, or notifications to prompt the user. The device also records whether the medication compartment has been accessed, which helps monitor patient adherence to the treatment plan. This collected data can be used by caregivers or healthcare

providers to analyze the patient's medication habits and activity levels. By combining monitoring, reminders, and activity encouragement, the system provides a supportive healthcare assistant for improving both physical and mental well-being.

2.2.2 Model Architecture

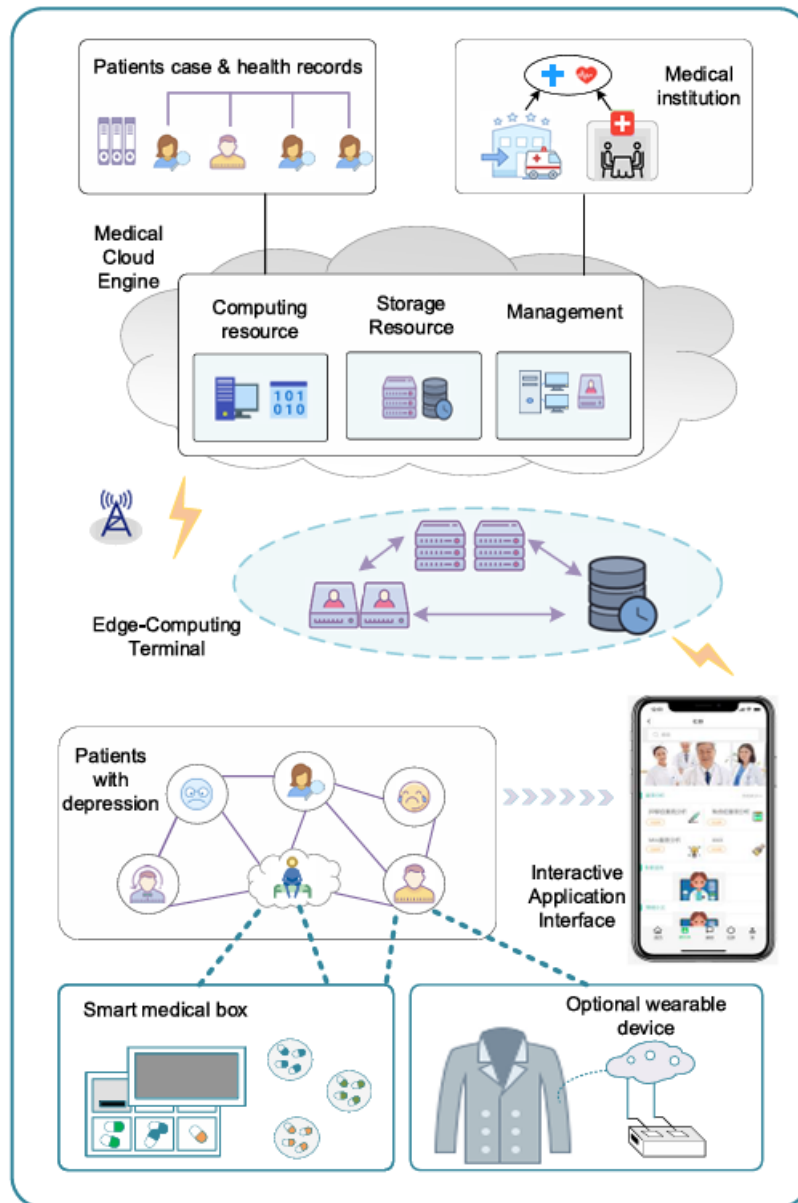


Figure 2.2: Architecture of MEMO Box Health Assistant System

The architecture of the MEMO Box system consists of several components including a microcontroller unit, medicine storage compartments, sensors, reminder modules, and a user interface. The microcontroller acts as the central controller that processes the data and manages the overall system operations. The medicine carrier stores pills in organized compartments for scheduled intake. Sensors and monitoring modules track user interaction with the device and ensure that medicines are taken at the correct time. Reminder systems such as alarms, LEDs, or notifications alert the user when medication or exercise activities are scheduled. The system may also communicate

with a mobile or monitoring platform to record patient activity and provide additional support for healthcare management.

Furthermore, the system architecture supports interaction between the hardware unit and the user through a simple interface. The microcontroller continuously monitors the scheduled medication times and controls the reminder system accordingly. When the alert is triggered, the user can access the medicine from the designated compartment in the MEMO Box. The system may also store usage data, which can be analyzed to understand the patient's adherence to medication and exercise routines. This integrated architecture ensures that both medication management and physical activity guidance work together to support the overall mental health of the patient.

2.2.3 Advantages

1. Helps patients with depression maintain regular medication schedules.
2. Encourages healthy habits by integrating exercise reminders with medication management.
3. Helps patients maintain regular medication schedules through automated reminders.
4. Encourages physical activity by providing exercise recommendations for improving mental health.
5. Supports monitoring of patient adherence to medication and daily routines.

2.2.4 Disadvantages

1. The system may depend heavily on electronic components which could fail or require maintenance.
2. Users who are not familiar with technology may find it difficult to operate the system.
3. The system may require continuous power and connectivity for effective operation.
4. The system depends on electronic components and power supply for proper functioning.
5. Patients who are not familiar with smart devices may find it difficult to use the system initially.

2.3 IoT-Based Medical Assistant for Efficient Monitoring of Patients in Response to COVID-19

The rapid spread of COVID-19 created a major challenge for hospitals and healthcare workers around the world. Doctors and nurses were required to monitor a large number of patients while also reducing direct contact to prevent infection. To address this issue, an IoT-based medical assistant system was proposed to monitor patients remotely and improve healthcare efficiency.

The proposed system collects important health parameters such as body temperature, heartbeat rate, and body movement using biomedical sensors connected to a microcontroller. The collected data is stored and automatically uploaded to a cloud platform so that doctors and nurses can monitor patient health remotely. If any abnormal condition is detected, the system generates an alert to notify the medical staff immediately.

The system also includes additional features such as monitoring the saline or blood level in the bottle, allowing patients to call nurses during emergencies, and providing medication reminders through an Android application. This integrated monitoring system helps reduce the workload of medical staff and ensures continuous monitoring of patients during critical situations such as the COVID-19 pandemic[3].

2.3.1 Methodology of IoT-Based Patient Monitoring System

The proposed system uses multiple biomedical sensors connected to an Arduino microcontroller to monitor the patient's health condition. Sensors such as the heartbeat sensor, LM35 temperature sensor, GY521 gyro sensor, and sonar sensor are used to measure heart rate, body temperature, patient movement, and saline level, respectively.

The sensor data is collected by the microcontroller and transmitted to a computer server where it is stored in an Excel file. The data is then automatically uploaded to a cloud storage platform such as OneDrive through the internet. This allows doctors and healthcare staff to monitor patient health data remotely from anywhere.

If the measured values exceed the normal range, an automated alert system notifies the doctor or nurse immediately. An Android application is also developed to provide medication reminders to patients and to allow them to call a nurse during emergencies. Additionally, the system enables patients to control the position of the hospital bed using servo motors connected to the microcontroller.

2.3.2 Model Architecture

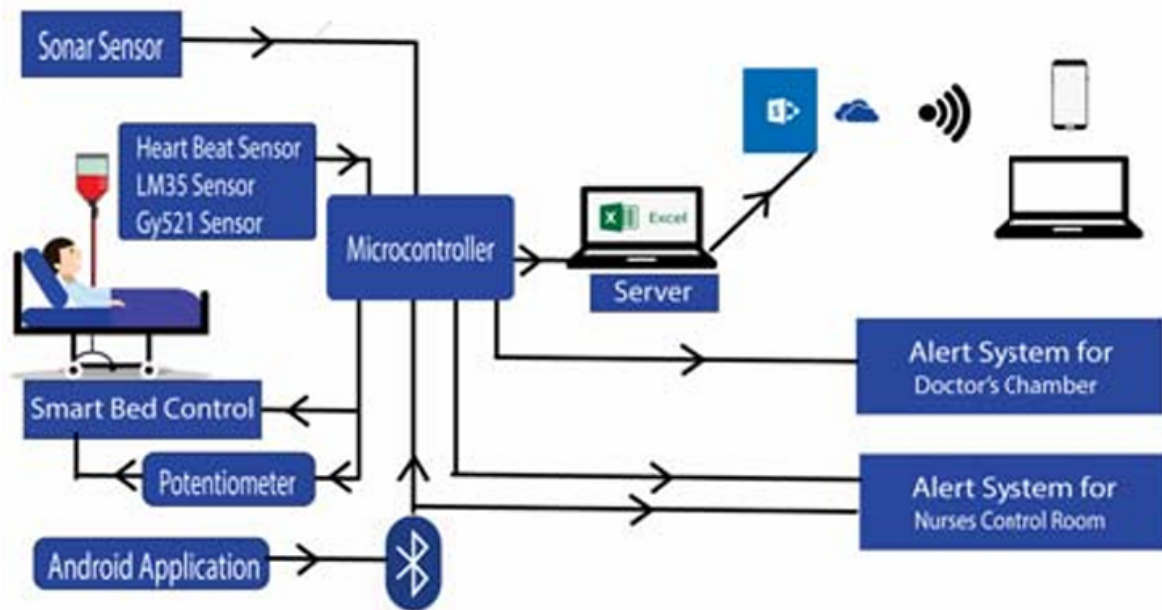


Figure 2.3: Architecture of IoT Based Patient Monitoring System

The architecture of the proposed system consists of hardware components, communication modules, and a cloud-based monitoring platform.

The main component of the system is the Arduino microcontroller which acts as the central controller for all sensors and devices. Biomedical sensors such as heartbeat sensors and temperature sensors are connected to the analog pins of the microcontroller, while movement and saline monitoring sensors are connected to the digital pins.

The sensor data is processed by the microcontroller and transmitted to a computer server using a USB interface. The server stores the collected data in an Excel file and uploads it to OneDrive so that healthcare professionals can access the data remotely through the internet.

The system also includes an alert mechanism that activates alarms when abnormal health conditions are detected. Servo motors are used to control the position of the patient's bed for comfort. Additionally, a Bluetooth module connects the system with an Android application that provides medication reminders, emergency nurse calls, and bed control features.

This integrated architecture enables continuous patient monitoring, remote healthcare supervision, and timely medical assistance.

2.3.3 Advantages

1. **Remote Patient Monitoring:** Doctors and nurses can monitor patient health conditions from a remote location through cloud-based data access.

2. **Reduced Risk of Infection:** The system minimizes direct contact between healthcare workers and infected patients.
3. **Continuous Health Monitoring:** Sensors continuously track important parameters such as heartbeat and body temperature.
4. **Emergency Alert System:** Automatic alerts notify doctors and nurses when abnormal health conditions occur.
5. **Medication Reminder:** The Android application reminds patients to take medicines at the correct time.

2.3.4 Disadvantages

1. **Hardware Complexity:** The system requires multiple sensors and electronic components which increases system complexity.
2. **Dependency on Internet:** Remote monitoring depends on stable internet connectivity.
3. **Maintenance Requirement:** Sensors and electronic devices require regular maintenance.
4. **Limited Testing:** The system was tested with a small number of patients which may affect generalization.
5. **Implementation Cost:** The use of multiple hardware components may increase the initial cost of implementation.

2.4 ACTIVA: Innovation in Quality of Care for Nursing Homes Through Activity Recognition

This paper presents ACTIVA, an innovative system designed to improve the quality of care in nursing homes through activity recognition and monitoring of elderly residents. The system uses wearable devices and environmental sensors to continuously monitor the activities and behavior of residents. Data collected from these devices is processed using a fog-cloud architecture that allows real-time monitoring and analysis. Caregivers can access this information through a mobile application, enabling them to detect abnormal behaviors and respond quickly to potential health issues. The system aims to enhance patient safety, improve healthcare efficiency, and support caregivers in nursing homes[4].

2.4.1 Methodology

The ACTIVA system is based on a multi-layer architecture that integrates wearable devices, environmental sensors, fog computing, and cloud services. Residents wear activity wristbands that collect motion and location data. Environmental sensors installed in rooms provide additional contextual information such as movement within specific areas.

The collected data is transmitted to local gateway devices and processed at the fog layer using Raspberry Pi devices. This local processing enables faster response times and reduces the amount of data transmitted to the cloud. The processed information is then sent to the cloud where further analysis and long-term storage are performed. Caregivers can monitor the residents through the ACTIVA mobile application, which provides alerts and activity reports.

2.4.2 Model Architecture

The architecture of the ACTIVA system follows a fog-cloud computing model. The system includes wearable activity wristbands worn by residents, environmental sensors placed in rooms, and anchor devices that help determine the location of residents within the nursing home. These devices communicate with gateway units that collect and forward the data to a fog computing layer.

The fog layer, implemented using Raspberry Pi devices, performs preliminary data processing and filtering. This allows real-time monitoring and faster detection of abnormal events. The processed data is then transmitted to the cloud layer where advanced analysis, data storage, and visualization are performed. Caregivers access this information through a mobile application that provides alerts, activity tracking, and health monitoring features.

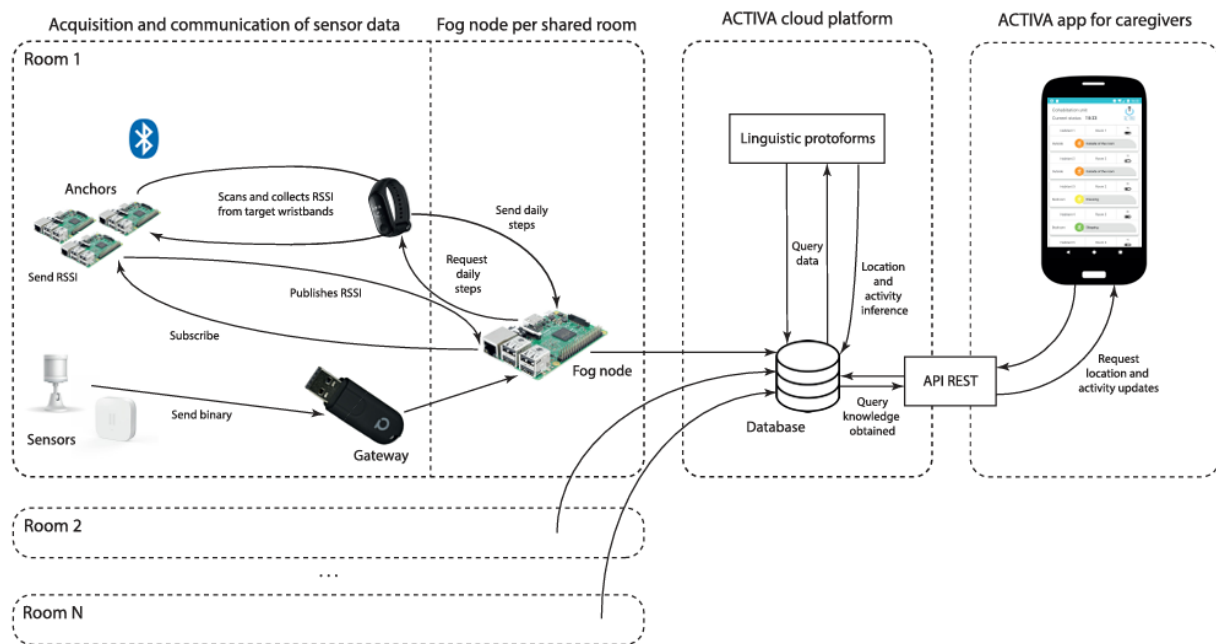


Figure 2.4: ACTIVA system architecture based on a fog-cloud architecture

2.4.3 Advantages

1. Provides continuous monitoring of elderly residents in nursing homes.
2. Uses a fog-cloud architecture to reduce latency and improve real-time processing.
3. Improves patient safety by detecting abnormal activities and behaviors.
4. Supports caregivers by providing alerts and activity reports through a mobile application.
5. Enables better healthcare management and decision making.

2.4.4 Disadvantages

1. Requires installation of multiple sensors and wearable devices.
2. Initial setup cost may be high for nursing homes.
3. Continuous monitoring may raise privacy concerns for residents.
4. System performance depends on stable network connectivity.

2.5 Voice Assistant for Health Self-Management for Older Adults

Health self-management is an important aspect of maintaining the well-being of older adults, especially those suffering from chronic diseases. Many elderly individuals face difficulties in remembering medication schedules, understanding medical information, and managing their daily health routines. Traditional health management tools such as mobile applications or written instructions are often difficult for older adults to use due to complex interfaces or physical limitations such as reduced vision and memory issues.

To address these challenges, a voice assistant-based health support system has been proposed. The system uses conversational voice interaction to help older adults understand medical information and follow their healthcare routines more effectively. By integrating a Large Language Model (LLM) with a voice assistant platform, the system can analyze medical summaries, answer health-related questions, and create personalized medication reminders. This approach improves accessibility and enables older adults to manage their health more independently[5].

2.5.1 Methodology

The system was developed using a multi-stage design process involving older adults. Initially, interviews were conducted with older individuals to identify the major challenges they face in health self-management. The study revealed two main problems: lack of awareness about their health information and difficulty in adhering to medical routines such as medication schedules.

Based on these findings, a prototype voice assistant was designed using a Large Language Model integrated with a commercial voice assistant platform. The system processes a patient's after-visit medical summary and provides a simple explanation of the important health information. It also allows users to ask questions related to their health condition and medications. Additionally, the system can automatically generate personalized medication reminders based on the patient's routine and preferences. The prototype was further refined through co-design workshops and usability testing with older adults to improve its effectiveness and usability.

Furthermore, the system development followed an iterative design approach where feedback from older adults was continuously incorporated to improve the functionality of the voice assistant. During the co-design workshops, participants interacted with the prototype and shared their experiences, suggestions, and difficulties while using the system. These insights helped the researchers refine the interaction flow, improve the clarity of health explanations, and design more flexible medication reminder settings. Finally, the refined system was evaluated through an in-home validation study to assess usability, effectiveness, and user satisfaction in real-life health self-management scenarios.

2.5.2 Model Architecture

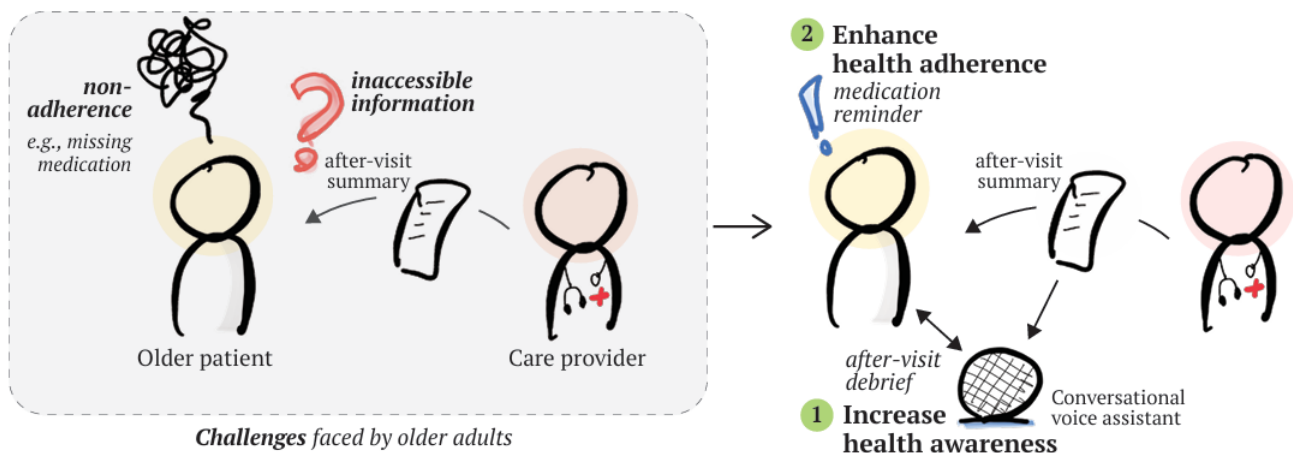


Figure 2.5: Architecture of Voice Assistant Health Self-Management System

The architecture of the proposed system consists of several key components including the user interface, voice assistant platform, Large Language Model, and healthcare data processing module. The user interacts with the system through voice commands using a smart speaker or voice-enabled device. The voice assistant processes the spoken input and forwards it to the Large Language Model for interpretation and response generation.

The system also receives medical data such as after-visit summaries from healthcare providers. The Large Language Model analyzes this information and generates simplified explanations that can be easily understood by the user. Based on the extracted medical information, the system can create medication reminders and provide health-related advice. The architecture also supports personalized interactions by adapting responses based on the user's routine, preferences, and health information.

The architecture also incorporates contextual information such as the user's daily routine, previous conversation history, and personal health preferences to provide more personalized assistance. When a user interacts with the system through voice commands, the assistant processes the request and consults the Large Language Model to generate meaningful responses. The system can answer health-related questions, explain medical instructions, and adjust reminder schedules based on the user's needs. This intelligent interaction allows the voice assistant to function as a supportive health companion, helping older adults stay informed about their health and maintain better adherence to their medical routines.

2.5.3 Advantages

1. Provides an easy and natural voice-based interaction for older adults.
2. Helps users understand complex medical information through simple explanations.
3. Improves medication adherence by generating personalized reminders.
4. Reduces the need for frequent consultation with healthcare providers for minor queries.
5. Supports independent health management for older adults.

2.5.4 Disadvantages

1. The system may raise privacy concerns related to sensitive health data.
2. Voice recognition errors may affect the accuracy of interactions.
3. Older adults unfamiliar with voice technology may require initial training.
4. The system depends on internet connectivity and voice assistant devices.
5. Incorrect interpretation of medical information could lead to misunderstandings.

Chapter 3

PROPOSED SYSTEM

The proposed system is an AI-powered Health Reminder and Emergency Alert Mobile Application designed to assist elderly individuals and patients in managing their medication schedules and health-related activities effectively. The system provides intelligent medicine reminders to ensure that patients take their medications at the correct time.

The application allows users to set reminders for different medicines along with dosage and timing details. When the scheduled time arrives, the system sends a notification to the patient reminding them to take the medicine. The patient can confirm the medicine intake within the application. If the medicine is not taken within the specified time, an alert notification is automatically sent to the registered caregiver.

The caregiver is also provided with access to the application through a separate login. The caregiver can monitor the patient's medication status, view missed doses, and receive alerts in case of emergencies. This helps caregivers take timely action when necessary.

The system also includes an emergency detection mechanism that can send alerts to the caregiver if unusual inactivity or emergency situations are detected. In addition, the application supports voice navigation, allowing elderly users to interact with the system using voice commands, making it easier for them to operate the application.

Another important feature of the proposed system is the appointment reminder module, which notifies patients about upcoming doctor appointments. The system can also provide a summary of the patient's activities, such as medicines taken, missed doses, and alerts generated.

Overall, the proposed system aims to improve patient safety, ensure timely medication intake, and provide continuous monitoring support through caregiver connectivity and intelligent alert mechanisms.

3.1 Process Overview

1. Project Planning

- **Define Project Scope and Objectives:** Outline the core functionalities, including intelligent medicine reminders, emergency detection, and caregiver support, to meet the needs of elderly and chronically ill users.
- **Identify Target Audience:** Focus on elderly users, chronically ill patients, and their caregivers, tailoring features to address accessibility and ease of use.
- **Establish Project Timeline and Milestones:** Create a timeline with milestones for design, development, testing, and deployment to ensure timely project completion.
- **Assemble Project Team:** Form a team with expertise in mobile app development, AI, sensor integration, user experience design, and cloud computing.

2. Requirements Gathering and Analysis

- **Gather User Requirements:** Conduct surveys and interviews with elderly users, caregivers, and healthcare providers to identify needs, such as voice navigation and emergency alerts.
- **Analyze User Feedback:** Evaluate feedback to prioritize features like multilingual support, intuitive UI, and real-time monitoring capabilities.
- **Define Functional and Non-functional Requirements:** Specify functional requirements (e.g., medicine reminders, emergency detection) and non-functional requirements (e.g., usability, scalability, security).

3. Design and Development

- **App Design:** Develop an intuitive, elderly-friendly user interface (UI) and user experience (UX) with features like large fonts, voice navigation, and simple navigation.
- **Sensor and AI Integration:** Implement motion sensors (accelerometer, gyroscope) for inactivity detection and AI algorithms for intelligent reminders and alert triggers.
- **Backend Development:** Integrate cloud based databases (e.g., Firebase, MongoDB) and RESTful APIs for real-time data syncing and caregiver dashboard updates.

- **Software Development:** Build the app using frameworks like Flutter or React Native, with languages such as Dart, JavaScript, or Kotlin, ensuring cross-platform compatibility.

4. Testing and Quality Assurance

- **Unit Testing:** Test individual modules, such as medicine reminders, voice navigation, and facial recognition, to ensure proper functionality.
- **Integration Testing:** Verify seamless interaction between modules, including sensor data processing and cloud syncing.
- **User Acceptance Testing (UAT):** Engage elderly users and caregivers to test the app's usability, accessibility, and reliability in real-world scenarios.
- **Performance Testing:** Assess the app's performance under various conditions, such as low battery, poor network connectivity, or high user load.

5. Deployment and Maintenance

- **App Store Deployment:** Publish the app on Google Play and Apple App Store, ensuring compliance with platform guidelines.
- **Monitoring and Support:** Monitor app performance, collect user feedback, and provide ongoing technical support to address issues.
- **Security Updates:** Regularly update the app to address security vulnerabilities and ensure data privacy for users and caregivers.
- **Feature Enhancements:** Introduce new features, such as advanced AI analytics or additional language support, based on user feedback and technological advancements.
- **User Training and Onboarding:** Provide tutorials, guides, and in app assistance to help elderly users and caregivers effectively navigate and utilize the app's features.

3.1.1 Process Flow Diagram

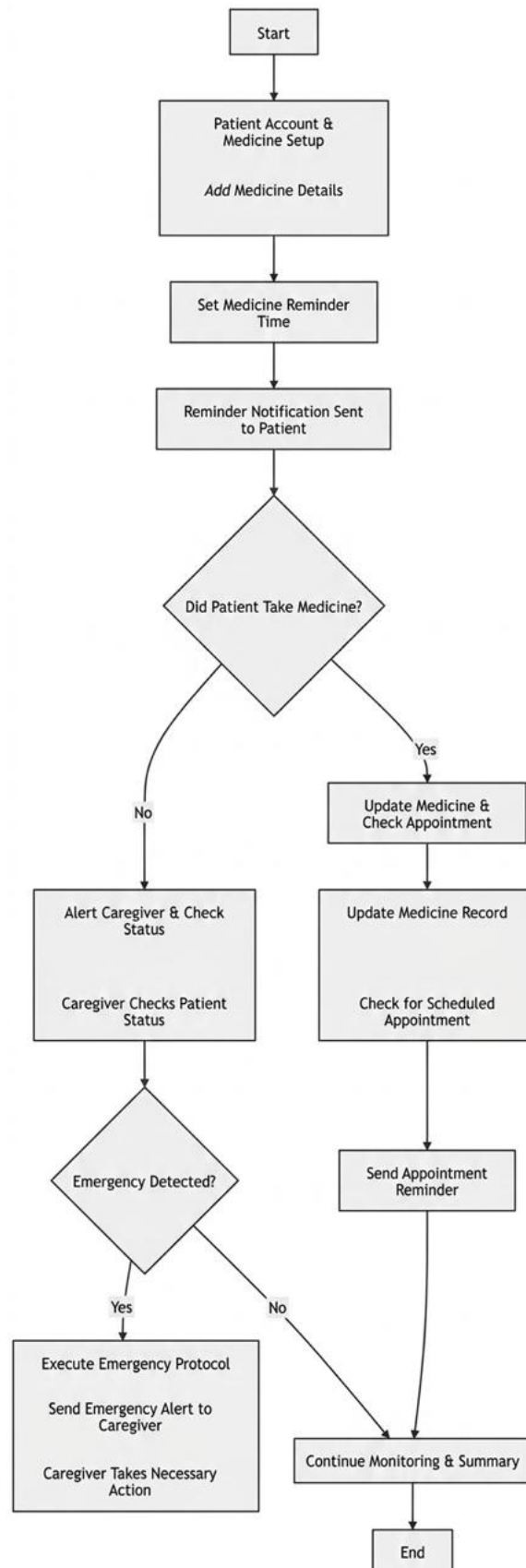


Figure 3.1: Process Flow Diagram

3.1.2 Architecture Diagram

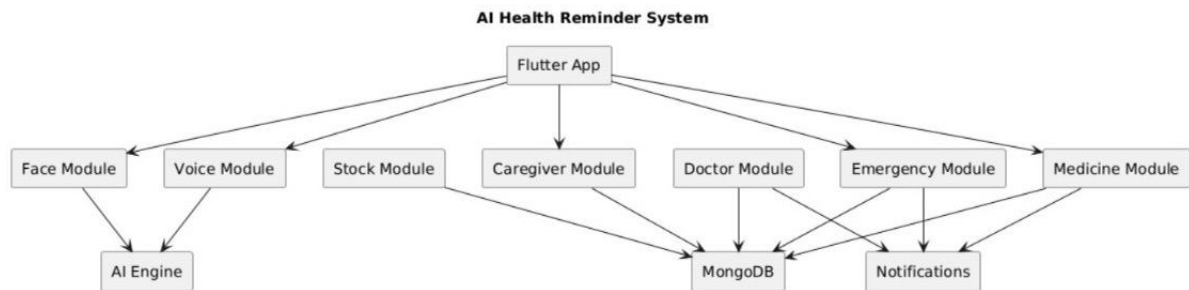


Figure 3.2: Architecture Diagram

3.1.3 Use case Diagram

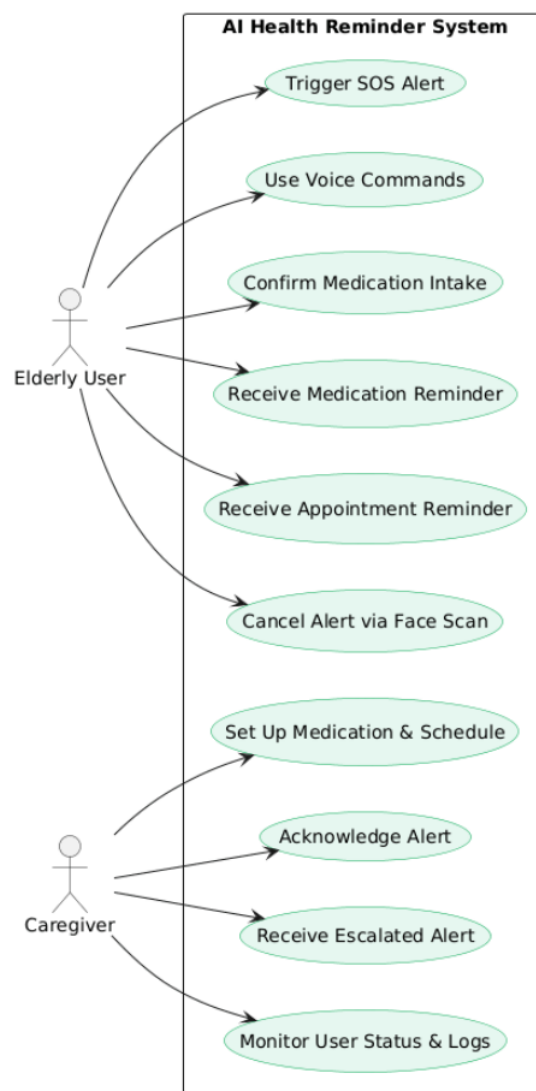


Figure 3.3: Use case diagram

3.1.4 Data flow diagram

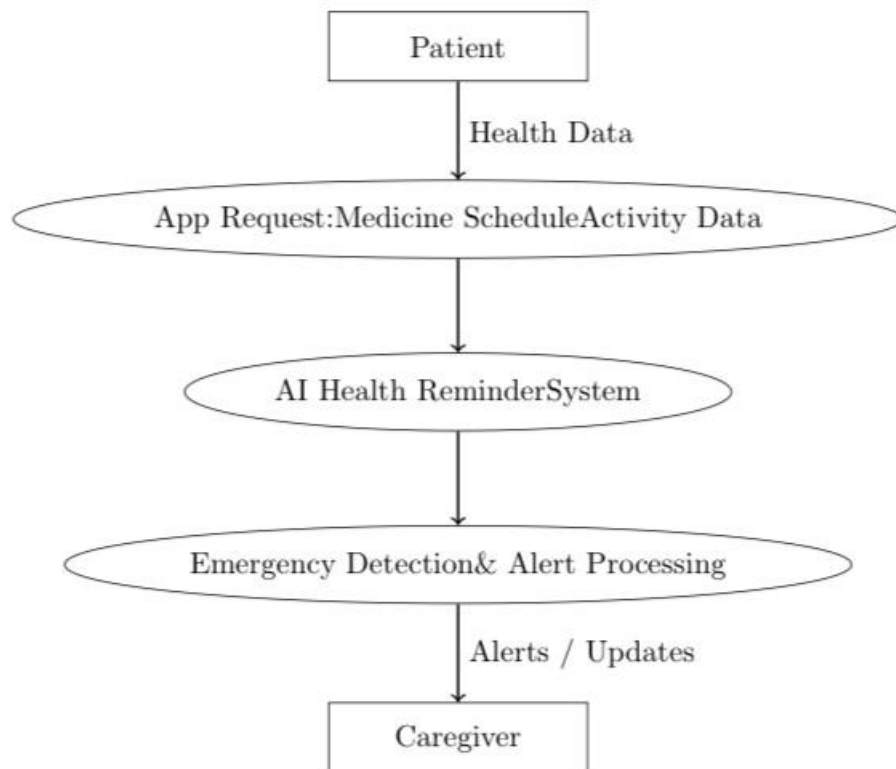


Figure 3.4: Data Flow Diagram Level 0

3.1.5 Data flow diagram

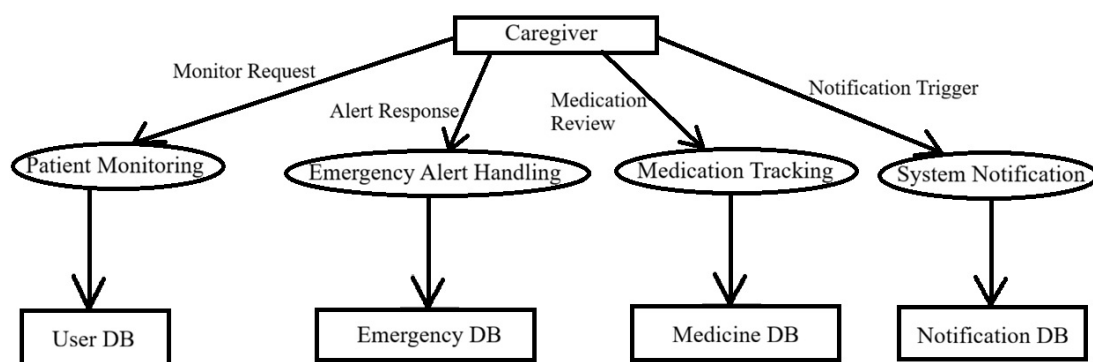


Figure 3.5: Data Flow Diagram Level 1-User

3.1.6 Data flow diagram

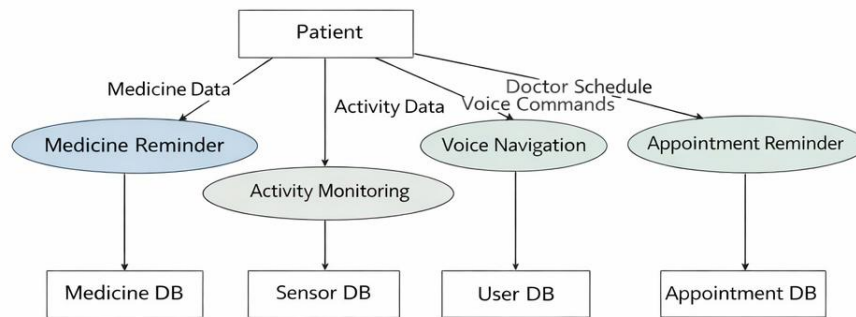


Figure 3.6: Data Flow Diagram Level 2-User

3.1.7 Class Diagram

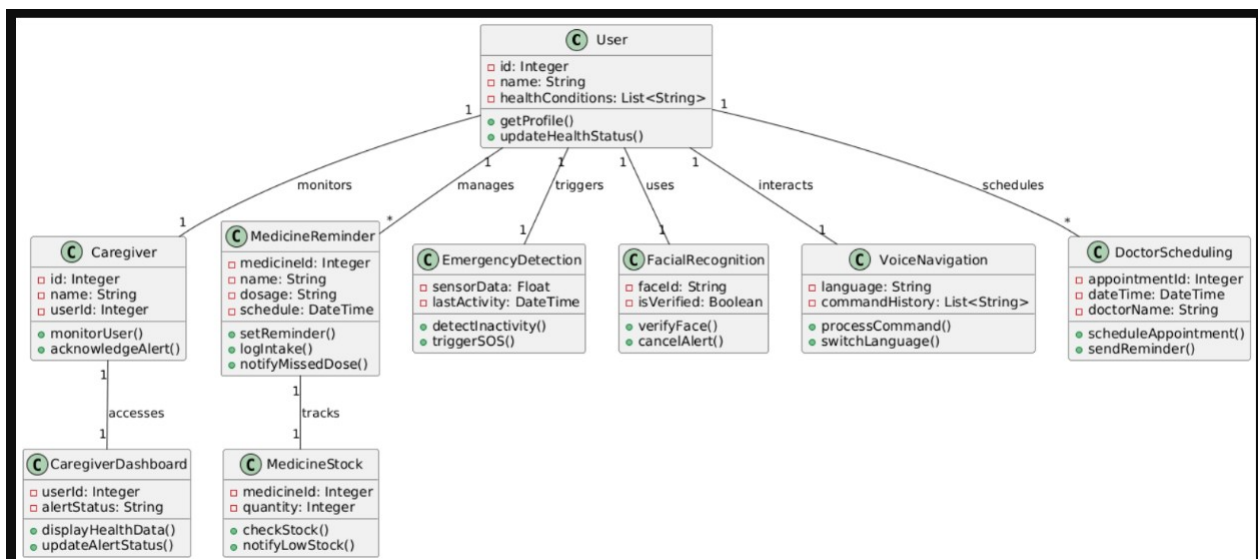


Figure 3.7: Class diagram

3.2 System Requirements

3.2.1 Hardware Requirements

The hardware requirements consist of a system with a Pentium 4 processor or above with a minimum main memory of 4GB RAM and a hard disk capacity of 40GB or above to support the development and execution of the application. Standard input devices such as a compatible mouse and keyboard are also required for proper user interaction with the system.

3.2.2 Software Requirements

The softwares are generally used for coding, development and execution of the system. The software platforms used in this system include JavaScript, HTML, CSS, Flutter, Python and MySQL Server. The development environment used for implementing the system includes Microsoft Windows 8 or above as the operating system along with development tools such as VSCode, Wamp or Xampp and SqlYog.

3.2.3 Flutter

Flutter is a cross-platform framework developed by Google for building natively compiled applications for mobile, web, and desktop from a single codebase. It uses the Dart programming language and offers a rich set of pre-built widgets for creating beautiful and responsive user interfaces. Flutter's hot reload feature allows developers to quickly see changes in their code without restarting the application, speeding up the development process. With its fast performance, expressive UI, and native-like experience across platforms, Flutter has gained popularity among developers for creating visually appealing and performant applications.

3.2.4 MySQL

MySQL is a widely used relational database management system that stores and manages application data efficiently. It uses structured query language (SQL) for managing and manipulating data stored in the database. MySQL provides high performance, reliability, and security for handling large amounts of structured data. It is commonly used in web and application development for storing user information, application records, and other system data.

3.2.5 Python

Python is a high-level, interpreted programming language known for its simplicity and readability. It is widely used for backend development, data processing, and application logic. Python provides a large number of libraries and frameworks that simplify development and enable efficient implementation of complex functionalities. Its flexibility and strong community support make it one of the most popular programming languages for modern software development.

3.3 Implementation Details

3.3.1 System Architecture

The proposed system is implemented using a client–server architecture in which the mobile application functions as the client while the backend server manages data processing, storage, and communication between the patient and caregiver. The application is designed specifically for Android devices to ensure better optimization and performance on Android smartphones, which are widely used by the target audience. The mobile application is developed using Flutter, which provides a flexible framework for building responsive and user-friendly mobile interfaces. The mobile application communicates with the backend server through APIs that enable data exchange between the mobile application and the database. The backend services are implemented using Python, which handles the core application logic such as medicine scheduling, reminder generation, emergency detection, and caregiver notification. All the data related to users, medicine schedules, alerts, and appointments are stored in a MySQL database to ensure structured storage and efficient retrieval of information.

3.3.2 Mobile Application Implementation

The mobile application is developed using Flutter to provide a smooth and responsive user experience for Android users. The interface is designed with simplicity and accessibility in mind so that elderly users can easily interact with the system. The application includes two types of users: patients and caregivers. The patient interface allows users to add and manage their medicine schedules by entering details such as medicine name, dosage, and timing. When the scheduled time arrives, the application generates a reminder notification prompting the patient to take the medicine. The patient can confirm medicine intake through the application, and this confirmation is recorded in the system. The application also provides voice navigation functionality that enables elderly users to interact with the application using voice commands, making the system easier to use for individuals who may have difficulty with traditional mobile interfaces. Additionally, the application includes an appointment reminder module that notifies users about upcoming doctor visits, helping them manage their healthcare schedules effectively. The application also monitors inactivity or emergency situations and triggers alerts when unusual behavior is detected.

3.3.3 Caregiver Monitoring Module

The caregiver module is implemented as a separate interface within the Android application, allowing caregivers to monitor the health activities of the patient. Through this module, caregivers can view the patient’s medication schedule and track whether medicines have been taken on time.

If a patient fails to confirm medicine intake within the specified time, the system automatically sends an alert notification to the caregiver. This enables caregivers to respond quickly and ensure that the patient follows the prescribed medication schedule. The caregiver module also provides access to patient activity summaries that include medicine intake history, missed doses, and emergency alerts. This monitoring capability ensures continuous support for patients who require assistance in managing their health routines.

3.3.4 Backend Implementation

The backend system is implemented using Python, which manages the communication between the mobile application and the database. The backend is responsible for handling user authentication, managing medicine schedules, generating reminders, and processing alerts. When a patient adds a medicine schedule through the mobile application, the information is sent to the backend server and stored in the database. The backend continuously monitors scheduled timings and triggers reminder notifications when required. If the patient does not confirm medicine intake within the defined time interval, the backend system automatically generates an alert notification that is sent to the caregiver. The backend also processes data related to emergency detection by analyzing sensor inputs from the mobile device, allowing the system to detect unusual inactivity or potential emergency situations. The mobile application communicates with the backend server through application programming interfaces (APIs) that enable data exchange between the application and the database.

3.3.5 Database Implementation

The system uses MySQL as the database management system to store and manage application data. The database maintains information related to patients, caregivers, medicine schedules, appointment reminders, and alert notifications. When users register or log in to the application, their information is securely stored in the database. Medicine details entered by the patient are also stored in structured tables so that reminder notifications can be generated accurately. The database also records patient activity data such as medicine confirmations, missed doses, and emergency alerts. This stored information allows caregivers to review patient health activities and helps the system generate summaries and reports when needed. MySQL ensures efficient data retrieval, data consistency, and reliable storage for the entire system.

3.3.6 Notification and Alert System

The notification and alert system is a crucial component of the application that ensures patients receive timely reminders and caregivers receive necessary alerts. When the scheduled time for medi-

cation arrives, the system sends a notification to the patient reminding them to take their medicine. If the patient confirms the intake, the system records the activity and updates the database. If the patient fails to respond within the allowed time period, the system automatically generates an alert notification that is sent to the caregiver. The system also generates notifications for upcoming doctor appointments so that patients do not miss important medical visits. In cases where unusual inactivity or potential emergency situations are detected, the system immediately sends an emergency alert to the caregiver to ensure prompt assistance.

3.3.7 Security Implementation

Security measures are implemented in the system to protect sensitive health data and ensure safe communication between different components of the application. User authentication mechanisms are used to ensure that only authorized users can access the system. Communication between the mobile application and backend server is secured to prevent unauthorized access to data. The system also implements role-based access control so that patients and caregivers can only access the features relevant to their roles. All user data stored in the database is handled securely to maintain the confidentiality and integrity of patient information.

3.3.8 Testing and Validation

After the implementation phase, the application undergoes multiple testing stages to ensure that all modules function correctly and efficiently. Unit testing is performed to verify the functionality of individual components such as medicine reminders, appointment notifications, and emergency alerts. Integration testing is conducted to ensure that the mobile application, backend server, and database work together seamlessly. User acceptance testing is carried out with elderly users and caregivers to evaluate the usability and accessibility of the system in real-world scenarios. Performance testing is also performed to assess the application's behavior under different conditions such as low battery levels, unstable network connectivity, and high user activity. These testing procedures help ensure that the system operates reliably and provides effective health monitoring support for patients and caregivers.

3.4 Methodology

3.4.1 Project Planning and Requirement Analysis

The methodology begins with project planning and requirement analysis, where the objectives and scope of the system are clearly defined. The primary aim of the system is to assist elderly individuals and patients in managing their medication schedules and health-related activities through

an intelligent reminder system and emergency alert mechanism. During this stage, the needs of potential users, such as patients and caregivers are carefully studied to determine the required features of the application. The system requirements include functionalities such as medicine reminders, caregiver notifications, emergency alerts, voice navigation, and appointment reminders. Since the application is specifically developed for Android users, the system design and development decisions are aligned with Android platform capabilities. Both functional and non-functional requirements, such as usability, reliability, security, and performance, are also defined to guide the development process.

3.4.2 System Design

After identifying the requirements, the system design phase focuses on defining the architecture and structure of the application. The system follows a client–server architecture where the Android mobile application acts as the client interface and communicates with backend services and the database. The application interface is designed to be simple and accessible so that elderly users can easily operate it. Features such as large fonts, simple navigation structures, and voice navigation are incorporated to improve usability. During this stage, the data flow between the Android application, backend services, and the database is designed to ensure efficient communication and proper synchronization of user data.

3.4.3 System Implementation

The implementation phase involves the development of the Android mobile application, backend services, and the database system using appropriate technologies. The mobile application is developed using Flutter, which enables efficient development of Android applications with a responsive and user-friendly interface. The backend of the system is implemented using Python, which handles the application logic, including user authentication, scheduling medicine reminders, generating notifications, and processing sensor data. The system also utilizes motion sensors available in Android devices to detect inactivity or potential emergency situations. All application data such as user details, medicine schedules, reminders, appointments, and alerts are stored and managed using the MySQL database management system.

3.4.4 Functional Module Development

Several functional modules are developed as part of the system implementation. The medicine reminder module allows patients to add medicines with dosage details and scheduled times. When the scheduled time arrives, the application generates a notification reminding the patient to take the medicine. The patient can confirm the medicine intake through the application. If the medicine

is not confirmed within the specified time, the system automatically sends an alert notification to the registered caregiver. The caregiver monitoring module allows caregivers to log in to the system and monitor patient activities, view medication records, and receive emergency alerts. The system also includes an appointment reminder module that notifies patients about upcoming doctor appointments. In addition, voice navigation functionality is included so that elderly users can interact with the application more conveniently.

3.4.5 Testing and Quality Assurance

Testing is performed to ensure that the Android application operates correctly and reliably. Unit testing is conducted to verify the functionality of individual modules such as medicine reminders, notifications, and voice navigation. Integration testing ensures that the Android application, back-end services, and database communicate properly with each other. User acceptance testing is carried out with potential users to evaluate the usability and accessibility of the application, particularly for elderly individuals. Performance testing is also performed to verify that the system functions effectively under different conditions, such as poor network connectivity, low battery levels, and multiple user interactions.

3.4.6 Deployment and Maintenance

After successful testing, the application is prepared for deployment for Android users. The application can be installed on Android devices, allowing patients and caregivers to use the system for managing medication schedules and monitoring health-related activities. Continuous monitoring and maintenance are performed to ensure the stable performance of the application. Updates and improvements are introduced based on user feedback and evolving technological requirements. Security mechanisms such as user authentication, secure data transmission, and role-based access control are also maintained to ensure that sensitive health-related information remains protected and confidential.

Chapter 4

CONCLUSION

The AI-Powered Health Reminder System and Emergency Alert Application provide an effective solution for assisting elderly individuals and patients in managing their medication schedules and healthcare activities. The system integrates intelligent medicine reminders, appointment notifications, emergency alert mechanisms, and caregiver monitoring features within a user-friendly Android mobile application. By allowing patients to set medicine schedules and confirm their intake while automatically notifying caregivers in case of missed doses or emergencies, the system enhances medication adherence and ensures timely medical support. Additional features such as voice navigation and activity monitoring improve accessibility and usability for elderly users, making the application easier to operate. The system also enables caregivers to remotely monitor patient activity, medication history, and alert notifications, thereby strengthening communication and supervision. Overall, the proposed system contributes to improving patient safety, promoting independent health management, and supporting caregivers in providing timely assistance, making it a practical and reliable healthcare support tool for elderly and chronically ill patients.

References

- [1] B. Ayshwarya and R. Velmurugan, “Intelligent and safe medication box in health iot platform for medication monitoring system with timely remainders,” in *2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS)*, vol. 1, pp. 1828–1831, 2021.
- [2] L. Luan, W. Xiao, K. Hwang, M. S. Hossain, G. Muhammad, and A. Ghoneim, “Memo box: Health assistant for depression with medicine carrier and exercise adjustment driven by edge computing,” *IEEE Access*, vol. 8, pp. 195568–195577, 2020.
- [3] R. Amin, T. S. Saha, M. F. B. Hassan, M. Anjum, and M. I. Tahmid, “Iot based medical assistant for efficient monitoring of patients in response to covid-19,” in *2020 2nd International Conference on Advanced Information and Communication Technology (ICAICT)*, pp. 83–87, 2020.
- [4] A. Montoro Lendínez, J. L. López Ruiz, C. Nugent, and M. Espinilla Estévez, “Activa: Innovation in quality of care for nursing homes through activity recognition,” *IEEE Access*, vol. 11, pp. 123335–123349, 2023.
- [5] A. Mahmood, S. Cao, M. Stiber, V. N. Antony, and C.-M. Huang, “Voice assistants for health self-management: Designing for and with older adults,” in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, CHI ’25, (New York, NY, USA), Association for Computing Machinery, 2025.
- [6] Z. Dong, J. Wei, X. Chen, and P. Zheng, “Face detection in security monitoring based on artificial intelligence video retrieval technology,” *IEEE Access*, vol. 8, pp. 63421–63433, 2020.
- [7] B. A. Motz, M. G. Mallon, and J. D. Quick, “Automated educative nudges to reduce missed assignments in college,” *IEEE Transactions on Learning Technologies*, vol. 14, no. 2, pp. 189–200, 2021.

- [8] Y. Huang, L.-Q. Tao, J. Yu, X. Li, Z. Wang, K. Zheng, and X. Chen, “Health monitoring and automatic notification device based on laser-induced graphene,” *IEEE Transactions on Electron Devices*, vol. 67, no. 10, pp. 4488–4492, 2020.
- [9] J. Qiu, L. Li, J. Sun, J. Peng, P. Shi, R. Zhang, Y. Dong, K. Lam, F. P.-W. Lo, B. Xiao, W. Yuan, N. Wang, D. Xu, and B. Lo, “Large ai models in health informatics: Applications, challenges, and the future,” *IEEE Journal of Biomedical and Health Informatics*, vol. 27, no. 12, pp. 6074–6087, 2023.
- [10] I. Agarwal, V. Sakthivel, and P. Prakash, “Toward inclusive healthcare: An llm-based multi-modal chatbot for preliminary diagnosis,” *IEEE Access*, vol. 13, pp. 136420–136432, 2025.
- [11] A. Mastrianni, A. Sarcevic, L. Chung, I. Zakeri, E. Alberto, Z. Milestone, I. Marsic, and R. S. Burd, “Designing interactive alerts to improve recognition of critical events in medical emergencies,” in *Proceedings of the 2021 ACM Designing Interactive Systems Conference*, DIS ’21, (New York, NY, USA), p. 864–878, Association for Computing Machinery, 2021.
- [12] I. Baek and L. Gualtieri, “Reimagining reminders to increase medication adherence in older adults,” in *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*, ACM MobiCom ’24, (New York, NY, USA), p. 2268–2274, Association for Computing Machinery, 2024.

Appendix A

SERVER

```
from datetime import datetime

from django.contrib import messages
from django.contrib.auth import authenticate, login
from django.contrib.auth.hashers import make_password
from django.contrib.auth.models import Group
from django.core.files.storage import FileSystemStorage
from django.http import JsonResponse
from django.shortcuts import render, redirect

# Create your views here.
from myapp.models import *

def index(request):
    return render(request, 'index.html')

def login_get(request):
    return render(request, 'login.html')

def login_post(request):
    uname=request.POST['uname']
    pwd=request.POST['pwd']
    user=authenticate(request, username=uname, password=pwd)
```

```
print("dddddddddddddd",user)
if user is not None:
    print("swssssss",user.groups)
    login(request, user)
    if user.groups.filter(name='Admin').exists():
        print("dddddddddddddddddddbkjvknkdjn")
        messages.success(request,'Admin Home')
        return redirect('/myapp/admin_home/')

    else:
        messages.success(request,'invalid User')
        return redirect('/myapp/login_get')
else:
    messages.success(request, 'invalid Username and Password')
    return redirect('/myapp/login_get')

def admin_home(request):
    return render(request,'admin_home.html')

def admin_view_caretaker(request):
    p=FamilyMember.objects.all()
    return render(request,'admin_view_caretaker.html',{'data':p})
def admin_view_patients(request,id):
    o=Patient.objects.filter(FAMILYMEMBER_id=id)
    return render(request,'admin_view_patients.html',{'data':o})
def admin_view_medicine(request,id):
    o=Medicine.objects.filter(PATIENT_id=id)

    return render(request,'admin_view_medicine.html',{'data':o})
def view_medication_plan(request,id):
    o=MedicinePlan.objects.filter(MEDICINE_id=id)
    return render(request,'admin_view_medicine_plan.html',{'data':o})

def admin_view_rating(request):
    o = review.objects.all()
```

```
total_reviews = review.objects.count()
average_rating = review.objects.aggregate(avg=Avg('rate'))['avg'] or 0
# this_month = review.objects.filter(date__month=datetime.now().month).count()

return render(request, 'admin_view_rate.html', {
    "data": o,
    "total_reviews": total_reviews,
    "average_rating": round(average_rating, 1),
    # "this_month": this_month,
})

def user_login(request):
    username = request.POST['username']
    password = request.POST['password']

    print(username, password)

    user = authenticate(request, username=username, password=password)
    if user is not None:

        if user.groups.filter(name='Patient').exists():
            loginid = user.id
            Patients = Patient.objects.get(LOGIN_id=loginid)

            sname = f"{Patients.NAME}"
            photo = str(Patients.face_image)
            print("oooooooooooo", sname, photo)
            return JsonResponse({
                "status": "ok",
                "lid": user.id,
                "name": sname,
                "photo": photo,
                "type": "Patient"
            })

        if user.groups.filter(name='Caretaker').exists():
            loginid = user.id
```

```
Caretaker = FamilyMember.objects.get(LOGIN_id=loginid)

sname = f"{Caretaker.NAME}"
print("oooooooooooo", sname)
return JsonResponse({
    "status": "ok",
    "lid": user.id,
    "name": sname,
    "type": "Caretaker"

})

else:
    return JsonResponse({"status": "no"})
else:
    return JsonResponse({"status": "no"})

def Caregiverregister(request):
    name = request.POST['name']
    address = request.POST['address']
    phone = request.POST['phone_number']
    gender=request.POST['gender']
    email = request.POST['email']
    password = request.POST['password']

    l = User.objects.create(username=email, password=str(make_password(password)))
    l.groups.add(Group.objects.get(name='Caretaker'))
    l.save()
    p = FamilyMember(NAME=name, ADDRESS=address, EMAIL=email, PHONE_NUMBER=phone,
    GENDER=gender, LOGIN_id=l.id)
    p.save()
    return JsonResponse({"status": "True"})

def Userregister(request):
    lid=request.POST['lid']
    cid=FamilyMember.objects.get(LOGIN_id=lid)
    careid=cid.id
    name = request.POST['name']
```

```

address = request.POST['address']
phone = request.POST['phone_number']
dob=request.POST['date_of_birth']
gender=request.POST['gender']
email = request.POST['email']
relationship=request.POST['relationship']
password = request.POST['password']
age=request.POST['age']
photo = request.FILES["profile_image"]
fs = FileSystemStorage()
date = datetime.now().strftime('%Y%m%d-%H%M%S') + '.jpg'
fs.save(date, photo)
path = fs.url(date)
l = User.objects.create(username=email, password=str(make_password(password)))
l.groups.add(Group.objects.get(name='Patient'))
l.save()
p = Patient(NAME=name, ADDRESS=address, EMAIL=email, PHONE_NUMBER=phone,
face_image=path,GENDER=gender,DATE_OF_BIRTH=dob, LOGIN_id=l.id,
FAMILYMEMBER_id=careid,relationship=relationship,AGE=age)
p.save()
return JsonResponse({"status": "True"})

```

```

from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt
from myapp.models import Patient, FamilyMember
import json

```

```

def view_patients(request):

```

```

    lid = request.GET['lid']
    cid = FamilyMember.objects.get(LOGIN_id=lid)
    careid = cid.id
    patients = Patient.objects.filter(FAMILYMEMBER_id=careid)

```

```
# Convert QuerySet to list of dictionaries
patients_list = []
for patient in patients:
    patients_list.append({
        'id': patient.id,
        'NAME': patient.NAME,
        'PHONE_NUMBER': patient.PHONE_NUMBER,
        'DATE_OF_BIRTH': patient.DATE_OF_BIRTH,
        'GENDER': patient.GENDER,
        'ADDRESS': patient.ADDRESS,
        'sleep_start_time': patient.sleep_start_time,
        'sleep_end_time': patient.sleep_end_time,
        'face_encoding': patient.face_encoding,
        'face_image': str(patient.face_image),
        'LOGIN_id': patient.LOGIN_id,
        'EMAIL': patient.EMAIL,
        'relationship': patient.relationship,
        'FAMILYMEMBER_id': patient.FAMILYMEMBER_id,
    })
print("dddddddddd",patients_list)

return JsonResponse({"status": "ok", "data": patients_list})

def delete_patient(request):
    patient_id=request.POST['patient_id']
    p=Patient.objects.get(id=patient_id)
    p.delete()
    return JsonResponse({"status":"ok"})

def add_medicine(request):
    name=request.POST['name']
    pid=request.POST['pid']
    medicine_type=request.POST['medicine_type']
    description=request.POST['description']
    dosage_strength=request.POST['dosage_strength']
    stock_quantity=request.POST['stock_quantity']
    i=Medicine(name=name,dosage_strength=dosage_strength,description=description,
    medicine_type=medicine_type,PATIENT_id=pid,stock_quantity=stock_quantity)
```

```
i.save()

return JsonResponse({"status": "ok"})

def view_medicines(request):
    pid=request.GET['pid']
    o=Medicine.objects.filter(PATIENT_id=pid)
    medicine_list = []
    for medicine in o:
        medicine_list.append({
            'id': medicine.id,
            'name': medicine.name,
            'dosage_strength': medicine.dosage_strength,
            'description': medicine.description,
            'medicine_type': medicine.medicine_type,
            'stock_quantity': medicine.stock_quantity,
            'low_stock_threshold': medicine.low_stock_threshold,

        })
    print("dddddddddd", medicine_list)

    return JsonResponse({"status": "ok", "data": medicine_list})

def update_medicine(request):
    medicine_id=request.POST['medicine_id']
    o=Medicine.objects.get(id=medicine_id)
    o.name=request.POST['name']
    o.medicine_type = request.POST['medicine_type']
    o.description = request.POST['description']
    o.dosage_strength = request.POST['dosage_strength']
    o.stock_quantity = request.POST['stock_quantity']
    o.save()
    return JsonResponse({"status": "ok"})

def delete_medicine(request):
    medicine_id=request.POST['medicine_id']
    o=Medicine.objects.get(id=medicine_id)
    o.delete()
    return JsonResponse({"status": "ok"})
```



```
def add_medicine_plan(request):
    patient_id=request.POST['patient_id']
    medicine_id=request.POST['medicine_id']
    dosage=request.POST['dosage']
    scheduled_time=request.POST['scheduled_time']
    frequency=request.POST['frequency']
    days_of_week=request.POST['days_of_week']
    is_active=request.POST['is_active']
    start_date=request.POST['start_date']
    end_date=request.POST['end_date']
    instructions=request.POST['instructions']
    p=MedicinePlan(PATIENT_id=patient_id,MEDICINE_id=medicine_id,dosage=dosage,
    scheduled_time=scheduled_time,frequency=frequency,days_of_week=days_of_week,
    is_active=is_active,start_date=start_date,
                    end_date=end_date,instructions=instructions)
    p.save()
    return JsonResponse({"status":"ok"})

def view_medicine_plans(request):
    patient_id=request.GET['patient_id']
    o=MedicinePlan.objects.filter(PATIENT_id=patient_id)
    medicineplan_list = []
    for medicine in o:
        medicineplan_list.append({
            'id': medicine.id,
            'medicine_name': medicine.MEDICINE.name,
            'dosage': medicine.dosage,
            'scheduled_time': medicine.scheduled_time,
            'frequency': medicine.frequency,
            'days_of_week': medicine.days_of_week,
            'start_date': medicine.start_date,
            'end_date':medicine.end_date,
            'instructions':medicine.instructions,
            'is_active':medicine.is_active

        })

    return JsonResponse({"status": "ok", "data": medicineplan_list})
```

```
def update_medicine_plan(request):
    plan_id=request.POST['plan_id']
    p=MedicinePlan.objects.get(id=plan_id)
    p.patient_id = request.POST['patient_id']
    p.MEDICINE_id = request.POST['medicine_id']
    p.dosage = request.POST['dosage']
    p.scheduled_time = request.POST['scheduled_time']
    p.frequency = request.POST['frequency']
    p.days_of_week = request.POST['days_of_week']
    p.is_active = request.POST['is_active']
    p.start_date = request.POST['start_date']
    p.end_date = request.POST['end_date']
    p.instructions = request.POST['instructions']
    p.save()
    return JsonResponse({"status":"ok"})

def delete_medicine_plan(request):
    plan_id=request.POST['plan_id']
    p=MedicinePlan.objects.get(id=plan_id)
    p.delete()
    return JsonResponse({"status":"ok"})

def doctor_app_add(request):
    PATIENT_id=request.POST['patient_id']
    doctor_name=request.POST['doctor_name']
    doctor_specialization=request.POST['doctor_specialization']
    hospital_name=request.POST['hospital_name']
    appointment_date=request.POST['appointment_date']
    appointment_time=request.POST['appointment_time']
    contact_phone=request.POST['contact_phone']
    purpose=request.POST['purpose']
    notes=request.POST['notes']
    o=Appointment(PATIENT_id=PATIENT_id,doctor_name=doctor_name,
    doctor_specialization=doctor_specialization,hospital_name=hospital_name,
    appointment_date=appointment_date,appointment_time=appointment_time,
```

```
contact_phone=contact_phone,purpose=purpose,notes=notes)
o.save()
return JsonResponse({"status":"ok"})

def view_appointments(request):
    patient_id=request.GET['patient_id']
    o=Appointment.objects.filter(PATIENT_id=patient_id)
    appo_list = []
    for appo in o:
        appo_list.append({
            'id': appo.id,
            'doctor_name': appo.doctor_name,
            'doctor_specialization': appo.doctor_specialization,
            'hospital_name': appo.hospital_name,
            'appointment_date': appo.appointment_date,
            'appointment_time': appo.appointment_time,
            'contact_phone': appo.contact_phone,
            'purpose':appo.purpose,
            'notes':appo.notes,

        })

    return JsonResponse({"status": "ok", "data": appo_list})

def delete_appointment(request):
    appointment_id=request.POST['appointment_id']
    p=Appointment.objects.get(id=appointment_id)
    p.delete()
    return JsonResponse({"status":"ok"})

def update_appointment(request):
    appointment_id = request.POST['appointment_id']
    p = Appointment.objects.get(id=appointment_id)
    p.doctor_name = request.POST['doctor_name']
    p.doctor_specialization = request.POST['doctor_specialization']
    p.hospital_name = request.POST['hospital_name']
    p.appointment_date = request.POST['appointment_date']
    p.appointment_time = request.POST['appointment_time']
    p.contact_phone = request.POST['contact_phone']
```

```
p.purpose = request.POST['purpose']
p.notes = request.POST['notes']
p.save()
return JsonResponse({"status":"ok"})

def get_emergency_contacts(request):
    try:
        patient_id = request.GET['patient_id']
        p=Patient.objects.get(LOGIN_id=patient_id)
        pid=p.id
        contacts = EmergencyContact.objects.filter(patient_id=pid)
        contacts_list = list(contacts.values('id', 'name', 'phone_number',
        'relationship', 'patient_id'))
        print("ddddddddddddd",contacts_list)
        return JsonResponse({'status': 'success', 'contacts': contacts_list})
    except Exception as e:
        return JsonResponse({'status': 'error', 'message': str(e)})

def add_emergency_contact(request):
    try:
        contact = EmergencyContact.objects.create(
            name=request.POST['name'],
            phone_number=request.POST['phone_number'],
            relationship=request.POST['relationship'],
            patient_id=request.POST['patient_id']
        )
        return JsonResponse({'status': 'success', 'contact_id': contact.id})
    except Exception as e:
        return JsonResponse({'status': 'error', 'message': str(e)})

def update_emergency_contact(request):
    contact = EmergencyContact.objects.get(id=request.POST['contact_id'])
    print("ddddddddddddd",contact)
```

```
contact.name = request.POST['name']
contact.phone_number = request.POST['phone_number']
contact.relationship = request.POST['relationship']
contact.save()
return JsonResponse({'status': 'success'})
```

```
def delete_emergency_contact(request):
    try:
        contact_id = request.POST['contact_id']
        contact = EmergencyContact.objects.get(id=contact_id)
        contact.delete()
        return JsonResponse({'status': 'success'})
    except Exception as e:
        return JsonResponse({'status': 'error', 'message': str(e)})
```

```
from datetime import datetime, date

def today_medicines(request):
    lid = request.GET['patient_id']
    print("Step 1: Received patient login ID:", lid)

    li = Patient.objects.get(LOGIN_id=lid)
    patient_id = li.id
    print("Step 2: Found patient with ID:", patient_id)

    # Get current date and day of week (1=Monday, 7=Sunday)
    today = date.today()
    today_weekday = today.weekday() + 1 # Convert to 1-7 format (Monday=1,
    Sunday=7)
    print("Step 4: Today's date:", today)
    print("Step 5: Today's weekday number (1-7):", today_weekday)

    # Get patient object
    patient = Patient.objects.get(id=patient_id)

    # Get all active medicine plans for the patient
```

```
today_plans = MedicinePlan.objects.filter(PATIENT=patient, is_active=True)
print("Step 7: Total active medicine plans found:", today_plans.count())

# Filter plans that are scheduled for today
today_medicines_list = []

for plan in today_plans:
    print(f"\n--- Processing Plan ID: {plan.id} ---")
    print(f"Plan details - Medicine: {plan.MEDICINE.name if plan.MEDICINE
    else
    'No Medicine'}")
    print(f"Dosage: {plan.dosage}, Time: {plan.scheduled_time}")

    # Parse days_of_week - handle both number formats and day names
    days_of_week_field = plan.days_of_week

    # Convert day names to numbers if needed
    day_name_to_number = {
        'Monday': '1', 'Tuesday': '2', 'Wednesday': '3',
        'Thursday': '4', 'Friday': '5', 'Saturday': '6', 'Sunday': '7'
    }

    # Check if the field contains day names instead of numbers
    if days_of_week_field in day_name_to_number:
        scheduled_days = [day_name_to_number[days_of_week_field]]
        print(f"Converted day name '{days_of_week_field}' to number:
        {scheduled_days}")
    elif days_of_week_field == 'Everyday':
        scheduled_days = ['1', '2', '3', '4', '5', '6', '7']
        print("Converted 'Everyday' to all days")
    elif days_of_week_field == 'Weekdays':
        scheduled_days = ['1', '2', '3', '4', '5']
        print("Converted 'Weekdays' to Monday-Friday")
    elif days_of_week_field == 'Weekends':
        scheduled_days = ['6', '7']
        print("Converted 'Weekends' to Saturday-Sunday")
```

```
else:
    # Assume it's already comma-separated numbers
    scheduled_days = [day.strip() for day in days_of_week_field.split
                      (',')]
    print(f"Parsed as comma-separated values: {scheduled_days}")

print(f"Checking if today ({today_weekday}) is in scheduled days:
{str(today_weekday) in scheduled_days}")

if str(today_weekday) in scheduled_days:
    print(f" Today ({today_weekday}) matches scheduled days")

# Check date range
try:
    # Handle date formats
    start_date_str = plan.start_date
    if start_date_str:
        if '-' in start_date_str:
            start_date = datetime.strptime(start_date_str,
            '%Y-%m-%d').
            date()
        else:
            print(f" Unexpected date format for start_date:
            {start_date_str}")
            continue
    else:
        print(" No start date - skipping")
        continue

    end_date = None
    if plan.end_date:
        if '-' in plan.end_date:
            end_date = datetime.strptime(plan.end_date,
            '%Y-%m-%d').
            date()
        else:
```

```
        print(f" Unexpected date format for end_date:
        {plan.end_date}")
        end_date = None

# Check if today is within the date range
if today >= start_date and (end_date is None or today <=
end_date):
    print(" Today is within valid date range")

# CRITICAL: Check if medicine has already been taken today
# Query the MedicationLog table to see if there's an entry
for today
taken_today = MedicationLog.objects.filter(
    MEDICINEPLAN_id=plan.id,
    taken_time__date=today # Check if taken on today's
    date).exists()

print(f" Checking if medicine was taken today:
{taken_today}")

if taken_today:
    print(f" Medicine {plan.MEDICINE.name} already taken
    today - skipping")
    continue # Skip this medicine if already taken
else:
    print(f" Medicine {plan.MEDICINE.name} not taken yet
    - adding to list")

# Get the latest medication log if exists (for tracking
missed medicines)
latest_log = MedicationLog.objects.filter(
    MEDICINEPLAN_id=plan.id
).order_by('-taken_time').first()

medicine_data = {
    'id': plan.id,
```



```
        'medicine_name': plan.MEDICINE.name if plan.MEDICINE else 'Unknown
        Medicine',
        'dosage': plan.dosage,
        'scheduled_time': plan.scheduled_time,
        'frequency': plan.frequency,
        'days_of_week': plan.days_of_week,
        'instructions': plan.instructions,
        'medicine_id': plan.MEDICINE.id if plan.MEDICINE
        else None,
        'medicine_type': plan
        .MEDICINE.medicine_type if plan.MEDICINE else '',
        'dosage_strength': plan.MEDICINE.dosage_strength
        if plan.MEDICINE else '',
        'is_taken': False, # Not taken yet
        'last_taken_time': latest_log.taken_time.strftime(
            '%Y-%m-%d %H:%M:%S') if latest_log else None,
    }
    today_medicines_list.append(medicine_data)
    print(f" Added to today's list. Current count: {len
    (today_medicines_list)}")
else:
    print(" Today is outside valid date range - not adding")
    if end_date and today > end_date:
        print(f" Reason: End date {end_date} is before
        today {today}")
    elif today < start_date:
        print(f" Reason: Start date {start_date} is after
        today {today}")
except Exception as e:
    print(f" Error parsing dates for plan {plan.id}: {e}")
    continue
else:
    print(f" Today ({today_weekday}) not in scheduled days - skipping")
```

```
print(f"\nStep 9: Total medicines for today after filtering:
```

```
{len(today_medicines_list)}")

# Sort by scheduled time
if today_medicines_list:
    today_medicines_list.sort(key=lambda x: x['scheduled_time'])
print("Step 10: Data after sorting by time:", today_medicines_list)

response_data = {
    'status': 'ok',
    'data': today_medicines_list,
    'count': len(today_medicines_list),
    'today_date': today.strftime('%Y-%m-%d'),
    'today_day': today.weekday
}
```

Appendix B

User Interface

```
import 'package:flutter/material.dart';
import 'package:health_app/lib/userregister.dart';
import 'package:health_app/lib/view_patient_details.dart';

import 'package:lottie/lottie.dart';
import 'package:shared_preferences/shared_preferences.dart';

import 'CaregiverProfilePage.dart';
import 'add_rate.dart';
import 'main.dart';

class CaregiverHomePage extends StatefulWidget {
  const CaregiverHomePage({super.key});

  @override
  State<CaregiverHomePage> createState() => _CaregiverHomePageState();
}

class _CaregiverHomePageState extends State<CaregiverHomePage> {
  String caregiverName = "Caregiver";
  int _currentIndex = 0;

  // Sample patient data
  final List<Map<String, dynamic>> _patients = [];

  @override
```

```
void initState() {
  super.initState();
  _loadCaregiverData();
}

Future<void> _loadCaregiverData() async {
  final prefs = await SharedPreferences.getInstance();
  setState(() {
    caregiverName = prefs.getString('uname') ?? 'Caregiver';
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Color(0xFFF8FDF8),
    appBar: AppBar(
      backgroundColor: Colors.white,
      foregroundColor: Color(0xFF1B5E20),
      elevation: 2,
      title: Text(
        'Dementia Care Dashboard',
        style: TextStyle(
          fontWeight: FontWeight.bold,
          color: Color(0xFF1B5E20),
          fontSize: 18,
        ),
      ),
      actions: [
        // IconButton(
        //   icon: Icon(Icons.notifications_active_rounded),
        //   onPressed: _showNotifications,
        //   color: Color(0xFFD32F2F),
        // ),
        PopupMenuButton<String>(
          icon: Icon(Icons.more_vert_rounded, color: Color(0xFF1B5E20)),

```

```
onSelected: (value) {  
  switch (value) {  
    case 'profile':  
      _viewProfile();  
      break;  
    case 'logout':  
      _logout();  
      break;  
  }  
},  
itemBuilder: (BuildContext context) => [  
  PopupMenuItem(  
    value: 'profile',  
    child: Row(  
      children: [  
        Icon(Icons.person_rounded, color: Color(0xFF1B5E20)),  
        SizedBox(width: 8),  
        Text('View Profile'),  
      ],  
    ),  
  ),  
  PopupMenuItem(  
    value: 'logout',  
    child: Row(  
      children: [  
        Icon(Icons.logout_rounded, color: Color(0xFFD32F2F)),  
        SizedBox(width: 8),  
        Text('Logout'),  
      ],  
    ),  
  ),  
],  
),  
body: _buildBody(),
```

```
bottomNavigationBar: _buildBottomNavigationBar(),  
floatingActionButton: FloatingActionButton(  
  onPressed: _addNewPatient,  
  backgroundColor: Color(0xFFD32F2F),  
  foregroundColor: Colors.white,  
  child: Icon(Icons.person_add_rounded),  
  shape: RoundedRectangleBorder(  
    borderRadius: BorderRadius.circular(16),  
  ),  
),  
);  
}
```

```
Widget _buildBody() {  
  switch (_currentIndex) {  
    case 0:  
      return _buildDashboard();  
    case 1:  
      return _buildPatientsList();  
    case 2:  
      return _buildMedications();  
    case 3:  
      return _buildEmergency();  
    default:  
      return _buildDashboard();  
  }  
}
```

```
Widget _buildDashboard() {  
  return SingleChildScrollView(  
    padding: EdgeInsets.all(16),  
    child: Column(  
      crossAxisAlignment: CrossAxisAlignment.start,  
      children: [  
        // Welcome Card  
        _buildWelcomeCard(),  
      ],  
    ),  
  );  
}
```

```
        SizedBox(height: 20),

        // Lottie Animation Section
        _buildLottieSection(),
        SizedBox(height: 20),

        // Recent Alerts
        // _buildRecentAlerts(),
        SizedBox(height: 20),

        // Upcoming Medications
        // _buildUpcomingMedications(),
      ],
    ),
  );
}

Widget _buildWelcomeCard() {
  return Card(
    elevation: 4,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(16),
    ),
    child: Container(
      width: double.infinity,
      padding: EdgeInsets.all(20),
      decoration: BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topLeft,
          end: Alignment.bottomRight,
          colors: [Color(0xFF1B5E20), Color(0xFF388E3C)],
        ),
        borderRadius: BorderRadius.circular(16),
      ),
      child: Row(
        children: [
```

```
Expanded(  
  child: Column(  
    crossAxisAlignment: CrossAxisAlignment.start,  
    children: [  
      Text(  
        'Welcome back,',  
        style: TextStyle(  
          color: Colors.white70,  
          fontSize: 16,  
        ),  
      ),  
      SizedBox(height: 4),  
      Text(  
        caregiverName,  
        style: TextStyle(  
          color: Colors.white,  
          fontSize: 24,  
          fontWeight: FontWeight.bold,  
        ),  
      ),  
      SizedBox(height: 8),  
      Text(  
        'You are making a difference in their lives every day',  
        style: TextStyle(  
          color: Colors.white70,  
          fontSize: 14,  
        ),  
      ),  
    ],  
  ),  
)  
Container(  
  width: 80,  
  height: 80,  
  decoration: BoxDecoration(  
    color: Colors.white.withOpacity(0.2),
```



```
        shape: BoxShape.circle,  
      ),  
      child: Icon(  
        Icons.medical_services_rounded,  
        color: Colors.white,  
        size: 40,  
      ),  
    ),  
  ],  
),  
);  
}
```

```
Widget _buildLottieSection() {  
  return Card(  
    elevation: 3,  
    shape: RoundedRectangleBorder(  
      borderRadius: BorderRadius.circular(16),  
    ),  
    child: Padding(  
      padding: EdgeInsets.all(16),  
      child: Column(  
        children: [  
          Text(  
            'Caregiving Support',  
            style: TextStyle(  
              fontSize: 18,  
              fontWeight: FontWeight.bold,  
              color: Color(0xFF1B5E20),  
            ),  
          ),  
          SizedBox(height: 8),  
          Text(  
            'Your dedication helps maintain their quality of life',  
            style: TextStyle(  

```

```
        color: Colors.grey[600],
        fontSize: 14,
      ),
      textAlign: TextAlign.center,
    ),
    SizedBox(height: 16),
    Container(
      height: 200,
      child: Lottie.asset(
        'assets/anil.json',
        fit: BoxFit.contain,
      ),
    ),
    SizedBox(height: 16),
    Text(
      'Remember: Patience and consistency are key in dementia care',
      style: TextStyle(
        color: Color(0xFFD32F2F),
        fontSize: 12,
        fontStyle: FontStyle.italic,
      ),
      textAlign: TextAlign.center,
    ),
  ],
),
);
}
```

```
Widget _buildRecentAlerts() {
  return Card(
    elevation: 3,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(16),
    ),
    child: Padding(
```

```
padding: EdgeInsets.all(16),
child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Row(
      children: [
        Icon(Icons.warning_amber_rounded, color: Color(0xFFFF57C0)),
        SizedBox(width: 8),
        Text(
          'Recent Alerts',
          style: TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
            color: Color(0xFF1B5E20),
          ),
        ),
      ],
    ),
    SizedBox(height: 12),
    _buildAlertItem(
      'Missed Medication',
      'John Smith missed his 2:00 PM medication',
      Icons.medication_outlined,
      Color(0xFFD32F2F),
    ),
    _buildAlertItem(
      'Emergency Alert',
      'Robert Brown activated emergency shake',
      Icons.emergency_rounded,
      Color(0xFFFF57C0),
    ),
  ],
),
);
}
```

```
Widget _buildAlertItem(String title, String subtitle, IconData icon, Color color) {  
  return ListTile(  
    leading: Container(  
      width: 40,  
      height: 40,  
      decoration: BoxDecoration(  
        color: color.withOpacity(0.1),  
        shape: BoxShape.circle,  
      ),  
      child: Icon(icon, color: color, size: 20),  
    ),  
    title: Text(  
      title,  
      style: TextStyle(  
        fontWeight: FontWeight.w600,  
        color: Colors.grey[800],  
      ),  
    ),  
    subtitle: Text(  
      subtitle,  
      style: TextStyle(  
        color: Colors.grey[600],  
        fontSize: 12,  
      ),  
    ),  
    trailing: Icon(Icons.chevron_right_rounded, color: Colors.grey[400]),  
    onTap: () {},  
  );  
}
```

```
Widget _buildUpcomingMedications() {  
  return Card(  
    elevation: 3,  
    shape: RoundedRectangleBorder(  
      borderRadius: BorderRadius.circular(16),  

```

```
),  
child: Padding(  
  padding: EdgeInsets.all(16),  
  child: Column(  
    crossAxisAlignment: CrossAxisAlignment.start,  
    children: [  
      Row(  
        children: [  
          Icon(Icons.schedule_rounded, color: Color(0xFF1B5E20)),  
          SizedBox(width: 8),  
          Text(  
            'Upcoming Medications',  
            style: TextStyle(  
              fontSize: 18,  
              fontWeight: FontWeight.bold,  
              color: Color(0xFF1B5E20),  
            ),  
          ),  
        ],  
      ),  
      SizedBox(height: 12),  
      _buildMedicationItem('John Smith', 'Donepezil 10mg', 'in 30 min'),  
      _buildMedicationItem('Mary Johnson', 'Memantine 20mg', 'in 2 hours'),  
      _buildMedicationItem('Robert Brown', 'Rivastigmine 6mg', 'in 4 hours'),  
    ],  
  ),  
),  
);  
}
```

```
Widget _buildMedicationItem(String patient, String medication, String time) {  
  return ListTile(  
    leading: Container(  
      width: 40,  
      height: 40,  
      decoration: BoxDecoration(  

```

```
        color: Color(0xFF1B5E20).withOpacity(0.1),
        shape: BoxShape.circle,
    ),
    child: Icon(Icons.medication_rounded, color: Color(0xFF1B5E20), size: 20),
),
title: Text(
    patient,
    style: TextStyle(
        fontWeight: FontWeight.w600,
        color: Colors.grey[800],
    ),
),
subtitle: Text(
    medication,
    style: TextStyle(
        color: Colors.grey[600],
    ),
),
trailing: Container(
    padding: EdgeInsets.symmetric(horizontal: 12, vertical: 6),
    decoration: BoxDecoration(
        color: Color(0xFFD32F2F).withOpacity(0.1),
        borderRadius: BorderRadius.circular(12),
    ),
    child: Text(
        time,
        style: TextStyle(
            color: Color(0xFFD32F2F),
            fontWeight: FontWeight.w600,
            fontSize: 12,
        ),
    ),
),
),
);
}
```

```
Widget _buildPatientsList() {  
  return Column(  
    children: [  
      Padding(  
        padding: EdgeInsets.all(16),  
        child: Text(  
          'Manage Patients',  
          style: TextStyle(  
            fontSize: 24,  
            fontWeight: FontWeight.bold,  
            color: Color(0xFF1B5E20),  
          ),  
        ),  
      ),  
      Expanded(  
        child: _buildPatientsContent(),  
      ),  
    ],  
  );  
}
```

```
Widget _buildPatientsContent() {  
  return SingleChildScrollView(  
    padding: EdgeInsets.all(20),  
    child: Column(  
      children: [  
        // Header Section  
        SizedBox(height: 30),  
  
        // Lottie Animation  
        Container(  
          height: 150,  
          child: Lottie.asset(  
            'assets/ani2.json', // You can use healthcare/patient  
            care related Lottie  
            fit: BoxFit.contain,
```

```
    ),  
    ),  
    SizedBox(height: 20),  
  
    // Features Grid  
    Text(  
      'Patient Care Features',  
      style: TextStyle(  
        fontSize: 22,  
        fontWeight: FontWeight.bold,  
        color: Color(0xFF1B5E20),  
      ),  
    ),  
    SizedBox(height: 8),  
    Text(  
      'Everything you need to provide exceptional dementia care',  
      style: TextStyle(  
        color: Colors.grey[600],  
        fontSize: 14,  
      ),  
      textAlign: TextAlign.center,  
    ),  
    SizedBox(height: 20),  
  
    // Features Grid  
    SizedBox(height: 30),  
  
    // Action Section  
    Container(  
      padding: EdgeInsets.all(25),  
      decoration: BoxDecoration(  
        color: Colors.white,  
        borderRadius: BorderRadius.circular(20),  
        boxShadow: [  
          BoxShadow(  

```



```
        color: Colors.grey.withOpacity(0.2),
        blurRadius: 20,
        offset: Offset(0, 10),
      ),
    ],
    border: Border.all(color: Colors.green.withOpacity(0.1)),
  ),
  child: Column(
    children: [
      Icon(
        Icons.medical_services_rounded,
        size: 50,
        color: Color(0xFF2E7D32),
      ),
      SizedBox(height: 15),
      Text(
        'Ready to Manage Patients?',
        style: TextStyle(
          fontSize: 20,
          fontWeight: FontWeight.bold,
          color: Color(0xFF1B5E20),
        ),
      ),
      // SizedBox(height: 10),
      // Text(
      //   'Access comprehensive patient management
      //   tools
      //   including medication tracking,
      //   emergency alerts, and health monitoring',
      //   style: TextStyle(
      //     color: Colors.grey[600],
      //     fontSize: 14,
      //   ),
      //   textAlign: TextAlign.center,
      // ),
      SizedBox(height: 20),
```

```
// Action Buttons

Row(
  children: [
    Expanded(
      child: ElevatedButton.icon(
        onPressed: _navigateToViewPatients,
        style: ElevatedButton.styleFrom(
          backgroundColor: Color(0xFF2E7D32),
          foregroundColor: Colors.white,
          padding: EdgeInsets.symmetric(vertical: 15),
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(12),
          ),
          elevation: 4,
        ),
        icon: Icon(Icons.visibility_rounded),
        label: Text(
          'View All Patients',
          style: TextStyle(
            fontWeight: FontWeight.w600,
            fontSize: 16,
          ),
        ),
      ),
    ),
  ],
),
 SizedBox(height: 10),

// Care Tips

],
),
```

```
);  
}  
  
Widget _buildFeatureCard({  
  required IconData icon,  
  required String title,  
  required String description,  
  required Color color,  
}) {  
  return Container(  
    decoration: BoxDecoration(  
      color: Colors.white,  
      borderRadius: BorderRadius.circular(12),  
      boxShadow: [  
        BoxShadow(  
          color: Colors.grey.withOpacity(0.1),  
          blurRadius: 10,  
          offset: Offset(0, 4),  
        ),  
      ],  
      border: Border.all(color: Colors.grey.withOpacity(0.2)),  
    ),  
    child: Padding(  
      padding: EdgeInsets.all(15),  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: [  
          Container(  
            width: 50,  
            height: 50,  
            decoration: BoxDecoration(  
              color: color.withOpacity(0.1),  
              shape: BoxShape.circle,  
            ),  
            child: Icon(icon, color: color, size: 24),  
          ),  
        ],  
      ),  
    ),  
  ),  
);
```

```
        SizedBox(height: 10),
        Text(
          title,
          style: TextStyle(
            fontWeight: FontWeight.bold,
            color: Color(0xFF1B5E20),
            fontSize: 14,
          ),
          textAlign: TextAlign.center,
        ),
        SizedBox(height: 5),
        Text(
          description,
          style: TextStyle(
            color: Colors.grey[600],
            fontSize: 11,
          ),
          textAlign: TextAlign.center,
        ),
      ],
    ),
  ),
);
}
```

```
Widget _buildStatItem(String label, String value, IconData icon) {
  return Column(
    children: [
      Container(
        width: 60,
        height: 60,
        decoration: BoxDecoration(
          color: Color(0xFF2E7D32).withOpacity(0.1),
          shape: BoxShape.circle,
        ),
        child: Icon(icon, color: Color(0xFF2E7D32), size: 24),
      ),
    ],
  );
}
```

```
    ),  
    SizedBox(height: 8),  
    Text(  
      value,  
      style: TextStyle(  
        fontSize: 18,  
        fontWeight: FontWeight.bold,  
        color: Color(0xFF1B5E20),  
      ),  
    ),  
  ),  
  Text(  
    label,  
    style: TextStyle(  
      color: Colors.grey[600],  
      fontSize: 12,  
    ),  
  ),  
],  
);  
}
```

```
Widget _buildCareTip(String tip) {  
  return Padding(  
    padding: EdgeInsets.symmetric(vertical: 6),  
    child: Row(  
      crossAxisAlignment: CrossAxisAlignment.start,  
      children: [  
        Icon(Icons.favorite_rounded, size: 16, color: Color(0xFFD32F2F)),  
        SizedBox(width: 8),  
        Expanded(  
          child: Text(  
            tip,  
            style: TextStyle(  
              color: Colors.grey[700],  
              fontSize: 13,  
            ),  
          ),  
        ],  
      ),  
    ),  
  );  
}
```

```
        ),  
      ),  
    ],  
  ),  
);  
}
```

```
Widget _buildMedications() {  
  return Container(  
    margin: EdgeInsets.all(16),  
    child: Column(  
      crossAxisAlignment: CrossAxisAlignment.start,  
      children: [  
        // Header Section  
        Container(  
          width: double.infinity,  
          padding: EdgeInsets.all(24),  
          decoration: BoxDecoration(  
            gradient: LinearGradient(  
              begin: Alignment.topLeft,  
              end: Alignment.bottomRight,  
              colors: [Color(0xFF2E7D32), Color(0xFF1B5E20)],  
            ),  
          borderRadius: BorderRadius.circular(20),  
          boxShadow: [  
            BoxShadow(  
              color: Color(0xFF2E7D32).withOpacity(0.3),  
              blurRadius: 15,  
              offset: Offset(0, 5),  
            ),  
          ],  
        ),  
        child: Column(  
          children: [  
            // Icon and Title  
            Container(  

```

```
        width: 80,
        height: 80,
        decoration: BoxDecoration(
          color: Colors.white.withOpacity(0.2),
          shape: BoxShape.circle,
        ),
        child: Icon(
          Icons.medication_rounded,
          color: Colors.white,
          size: 40,
        ),
      ),
      SizedBox(height: 16),
      Text(
        'Medicine Ratings & Reviews',
        style: TextStyle(
          fontSize: 24,
          fontWeight: FontWeight.bold,
          color: Colors.white,
        ),
        textAlign: TextAlign.center,
      ),
      SizedBox(height: 8),
      Text(
        'Share your experience and help others',
        style: TextStyle(
          fontSize: 16,
          color: Colors.white.withOpacity(0.9),
        ),
        textAlign: TextAlign.center,
      ),
    ],
  ),
),
```

```
// Features Section
```

```
// Features Grid
GridView.count(
  shrinkWrap: true,
  physics: NeverScrollableScrollPhysics(),
  crossAxisCount: 2,
  crossAxisSpacing: 12,
  mainAxisSpacing: 12,
  childAspectRatio: 0.8,
  children: [

    _buildFeatureCard(
      icon: Icons.people_rounded,
      title: 'Help Others',
      description: 'Share experiences with fellow patients',
      color: Colors.green,
    ),
    _buildFeatureCard(
      icon: Icons.feedback_rounded,
      title: 'Provide Feedback',
      description: 'Help improve healthcare quality',
      color: Colors.orange,
    ),

  ],
),
 SizedBox(height: 24),

// How It Works Section

SizedBox(height: 24),

// Testimonial Section
```



```
// Main Action Button
Container(
  width: double.infinity,
  child: ElevatedButton(
    onPressed: () {
      // Navigate to rating page
      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (_) => AppRatingPage(

            ),
          ),
        );
    },
    style: ElevatedButton.styleFrom(
      backgroundColor: Color(0xFF2E7D32),
      foregroundColor: Colors.white,
      padding: EdgeInsets.symmetric(vertical: 16, horizontal: 32),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
      ),
      elevation: 4,
    ),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Icon(Icons.star_rounded, size: 24),
        SizedBox(width: 12),
        Text(
          'START RATING MEDICINES',
          style: TextStyle(
            fontSize: 16,
            fontWeight: FontWeight.bold,
          ),
        ),
      ],
    ),
  ),
)
```

```
        ),
      ],
    ),
  ),
),
  SizedBox(height: 16),

],
),
);
}

// Widget _buildFeatureCard({
//   required IconData icon,
//   required String title,
//   required String description,
//   required Color color,
// }) {
//   return Container(
//     decoration: BoxDecoration(
//       color: Colors.white,
//       borderRadius: BorderRadius.circular(12),
//       boxShadow: [
//         BoxShadow(
//           color: Colors.black12,
//           blurRadius: 8,
//           offset: Offset(0, 2),
//         ),
//       ],
//     ),
//     child: Padding(
//       padding: const EdgeInsets.all(16),
//       child: Column(
//         mainAxisAlignment: MainAxisAlignment.center,
//         children: [
```

```
//      Container(  
//          width: 50,  
//          height: 50,  
//          decoration: BoxDecoration(  
//              color: color.withOpacity(0.1),  
//              shape: BoxShape.circle,  
//          ),  
//          child: Icon(  
//              icon,  
//              color: color,  
//              size: 24,  
//          ),  
//      ),  
//      SizedBox(height: 12),  
//      Text(  
//          title,  
//          style: TextStyle(  
//              fontSize: 14,  
//              fontWeight: FontWeight.bold,  
//              color: Color(0xFF1B5E20),  
//          ),  
//          textAlign: TextAlign.center,  
//      ),  
//      SizedBox(height: 8),  
//      Text(  
//          description,  
//          style: TextStyle(  
//              fontSize: 12,  
//              color: Colors.grey[600],  
//          ),  
//          textAlign: TextAlign.center,  
//      ),  
//      ],  
//      ),  
//      ),  
//      );
```

```
// }
```

```
Widget _buildStep({
  required int number,
  required String title,
  required String description,
}) {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 8),
    child: Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Container(
          width: 24,
          height: 24,
          decoration: BoxDecoration(
            color: Color(0xFF2E7D32),
            shape: BoxShape.circle,
          ),
          child: Center(
            child: Text(
              number.toString(),
              style: TextStyle(
                color: Colors.white,
                fontSize: 12,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
        ),
        SizedBox(width: 12),
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
```

```
        title,  
        style: TextStyle(  
          fontWeight: FontWeight.w600,  
          color: Color(0xFF1B5E20),  
        ),  
      ),  
      SizedBox(height: 2),  
      Text(  
        description,  
        style: TextStyle(  
          fontSize: 12,  
          color: Colors.grey[600],  
        ),  
      ),  
    ],  
  ),  
),  
],  
),  
),  
],  
),  
);  
}
```

```
Widget _buildEmergency() {  
  return Center(  
    child: Text(  
      'Emergency Alerts',  
      style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),  
    ),  
  );  
}
```

```
Widget _buildBottomNavigationBar() {  
  return BottomNavigationBar(  
    currentIndex: _currentIndex,  
    onTap: (index) {  
      setState(() {
```

```
        _currentIndex = index;
    });
},
type: BottomNavigationBarType.fixed,
selectedItemColor: Color(0xFFD32F2F),
unselectedItemColor: Colors.grey[600],
selectedLabelStyle: TextStyle(fontWeight: FontWeight.w600),
items: [
    BottomNavigationBarItem(
        icon: Icon(Icons.dashboard_rounded),
        label: 'Dashboard',
    ),
    BottomNavigationBarItem(
        icon: Icon(Icons.group_rounded),
        label: 'Patients',
    ),
    BottomNavigationBarItem(
        icon: Icon(Icons.star),
        label: 'Review',
    ),
    // BottomNavigationBarItem(
    //     icon: Icon(Icons.emergency_rounded),
    //     label: 'Emergency',
    // ),
],
);
}
```

```
Color _getStatusColor(String status) {
    switch (status.toLowerCase()) {
        case 'stable':
            return Color(0xFF1B5E20);
        case 'good':
            return Color(0xFF388E3C);
        case 'needs attention':
            return Color(0xFFD32F2F);
    }
}
```

```
        default:
            return Colors.grey;
    }
}

void _showNotifications() {
    // Implement notifications view
}

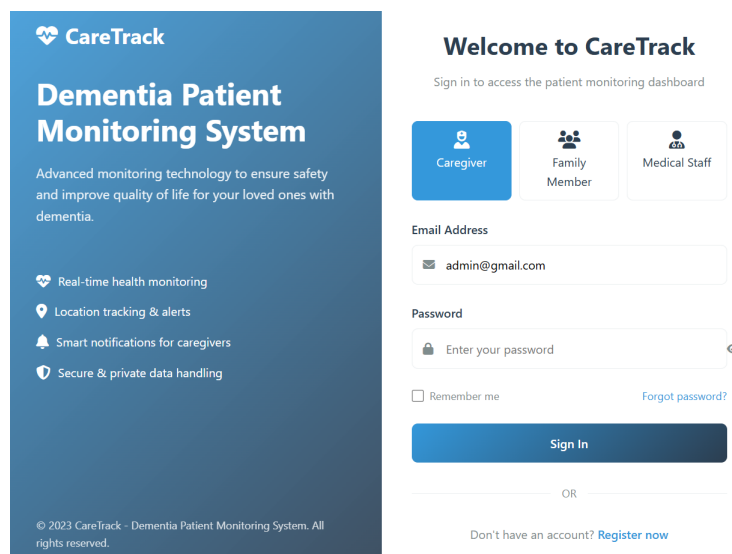
void _viewProfile() {
    Navigator.push(
        context,
        MaterialPageRoute(builder: (_) => CaregiverProfilePage()),
    );
}

void _logout() {
    showDialog(
        context: context,
        builder: (context) => AlertDialog(
            title: Text('Logout'),
            content: Text('Are you sure you want to logout?'),
            actions: [
                TextButton(
                    onPressed: () => Navigator.pop(context),
                    child: Text('Cancel'),
                ),
                TextButton(
                    onPressed: () {
                        Navigator.pop(context);
                        Navigator.pushReplacement(
                            context,
                            MaterialPageRoute(builder: (_) => IPPage(title: '')),
                        );
                    },
                    child: Text(
                        'Logout',
                        style: TextStyle(color: Color(0xFFD32F2F)),
                    ),
                ),
            ],
        ),
    );
}
```

```
        ),  
        ),  
    ],  
    ),  
);  
}  
  
void _addNewPatient() {  
    Navigator.push(  
        context,  
        MaterialPageRoute(builder: (_) => PatientRegisterPage())  
    );  
}  
  
void _navigateToViewPatients() {  
    Navigator.push(  
        context,  
        MaterialPageRoute(builder: (_) => ViewPatientsPage())  
    );  
}  
  
void _viewPatientDetails(Map<String, dynamic> patient) {  
    Navigator.push(  
        context,  
        MaterialPageRoute(builder: (_) => ViewPatientsPage())  
    );  
}  
}
```


Appendix C

Screenshots



The screenshot shows the 'Welcome to CareTrack' sign-in page. On the left, a blue sidebar contains the CareTrack logo, the title 'Dementia Patient Monitoring System', a description of the system, and a list of features: Real-time health monitoring, Location tracking & alerts, Smart notifications for caregivers, and Secure & private data handling. The main content area has three role-based login buttons: Caregiver (blue), Family Member (light blue), and Medical Staff (light blue). Below these are input fields for 'Email Address' (containing 'admin@gmail.com') and 'Password' (with a placeholder 'Enter your password'). There are checkboxes for 'Remember me' and a link for 'Forgot password?'. A large blue 'Sign In' button is at the bottom. Below the button is an 'OR' separator and a link to 'Register now' for users without an account.

Welcome to CareTrack

Sign in to access the patient monitoring dashboard

Caregiver **Family Member** **Medical Staff**

Email Address
admin@gmail.com

Password
Enter your password

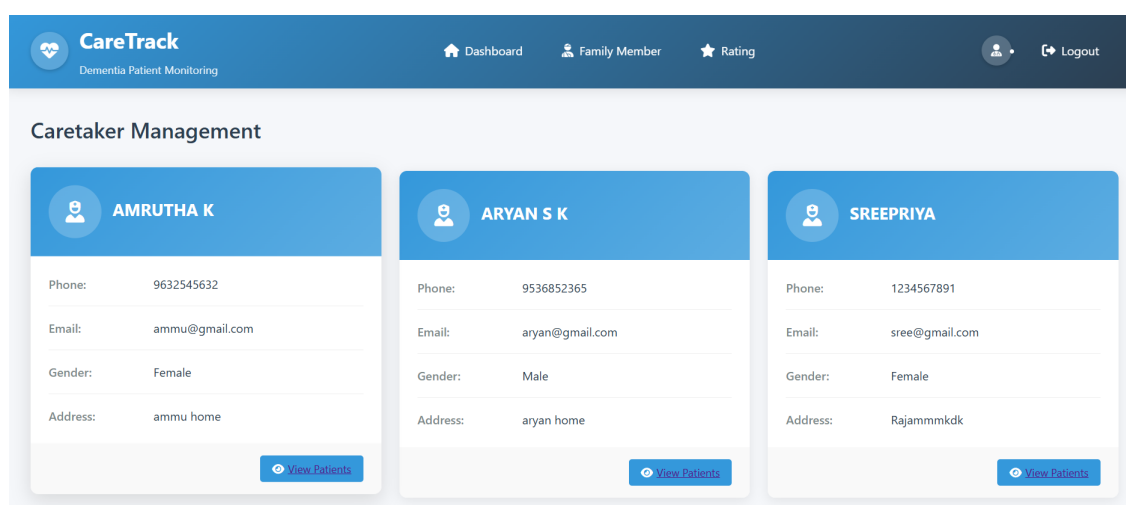
☐ Remember me [Forgot password?](#)

Sign In

OR

Don't have an account? [Register now](#)

Figure C.1: Sign-In Page



The screenshot shows the 'Caretaker Management' admin page. At the top is a dark blue header with the CareTrack logo, navigation links for 'Dashboard', 'Family Member', and 'Rating', and a 'Logout' button. The main content area is titled 'Caretaker Management' and displays three caregiver profiles in blue and white cards. Each card shows the caregiver's name, phone number, email, gender, and address, along with a 'View Patients' button.

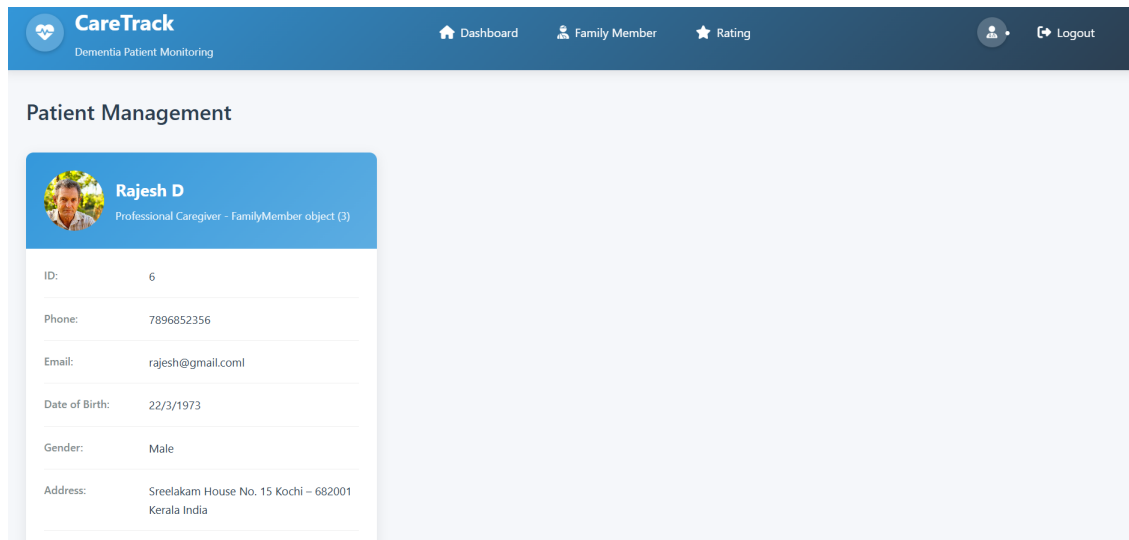
CareTrack Dementia Patient Monitoring

Dashboard Family Member Rating Logout

Caretaker Management

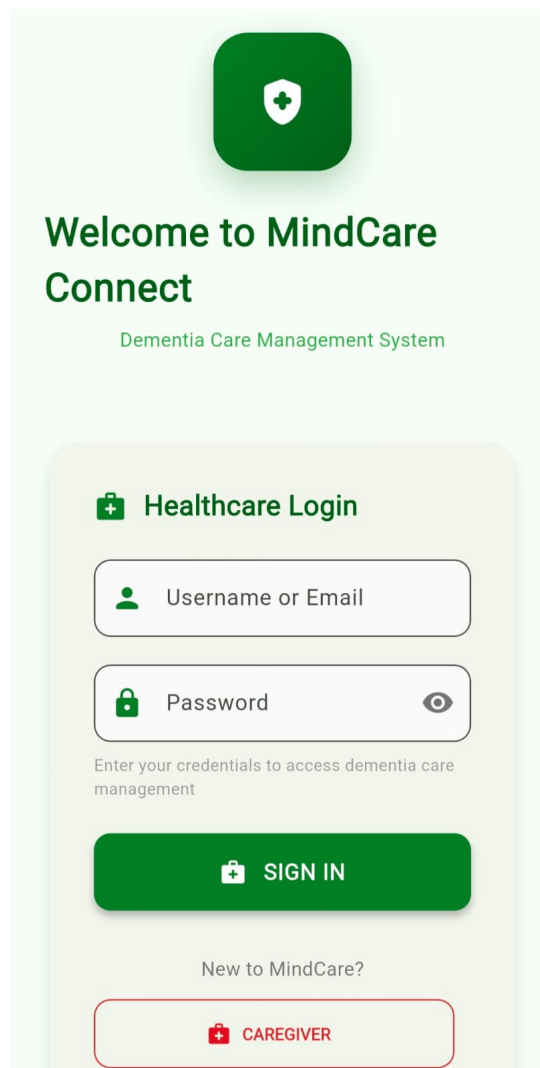
AMRUTHA K	ARYAN S K	SREEPRIYA
Phone: 9632545632	Phone: 9536852365	Phone: 1234567891
Email: ammu@gmail.com	Email: aryan@gmail.com	Email: sree@gmail.com
Gender: Female	Gender: Male	Gender: Female
Address: ammu home	Address: aryan home	Address: Rajammkdk
View Patients	View Patients	View Patients

Figure C.2: Admin Page



Rajesh D Professional Caregiver - FamilyMember object (3)	
ID:	6
Phone:	7896852356
Email:	rajesh@gmail.com
Date of Birth:	22/3/1973
Gender:	Male
Address:	Sreelakam House No. 15 Kochi – 682001 Kerala India

Figure C.3: Patient Management



Welcome to MindCare Connect
Dementia Care Management System

Healthcare Login

Username or Email

Password

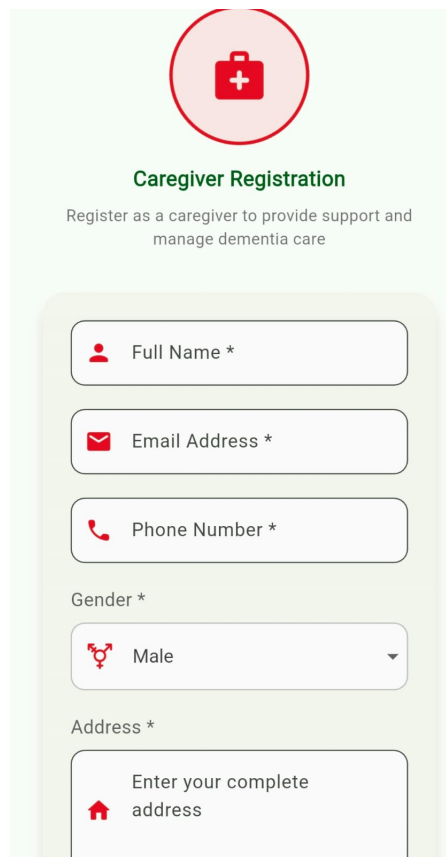
Enter your credentials to access dementia care management

SIGN IN

New to MindCare?

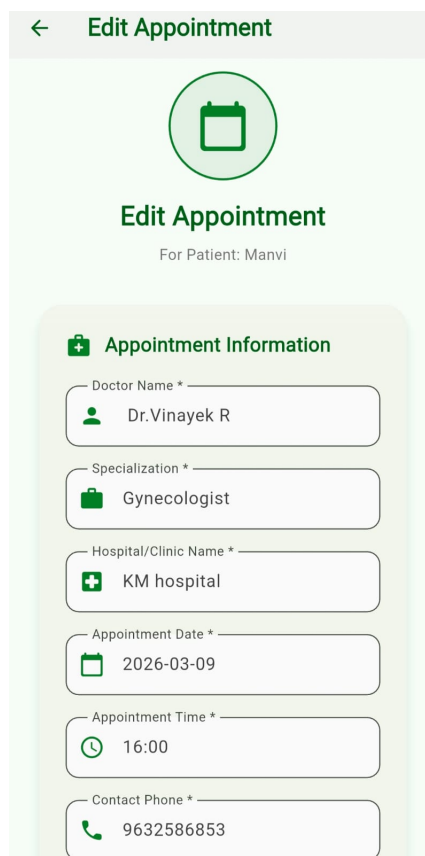
CAREGIVER

Figure C.4: Registration



The image shows a mobile app screen for 'Caregiver Registration'. At the top, there is a red circular icon with a white first aid kit. Below it, the title 'Caregiver Registration' is in green. A subtitle reads: 'Register as a caregiver to provide support and manage dementia care'. The form consists of several input fields: 'Full Name *' with a person icon, 'Email Address *' with an envelope icon, 'Phone Number *' with a phone icon, 'Gender *' with a dropdown menu showing 'Male' and a female icon, and 'Address *' with a house icon and the placeholder text 'Enter your complete address'.

Figure C.5: Caregiver Registration



The image shows a mobile app screen for 'Edit Appointment'. At the top, there is a green circular icon with a white calendar. Below it, the title 'Edit Appointment' is in green. A subtitle reads: 'For Patient: Manvi'. The form is titled 'Appointment Information' with a green plus icon. It contains several input fields: 'Doctor Name *' with a person icon and the value 'Dr.Vinayek R', 'Specialization *' with a green bag icon and the value 'Gynecologist', 'Hospital/Clinic Name *' with a green plus icon and the value 'KM hospital', 'Appointment Date *' with a green calendar icon and the value '2026-03-09', 'Appointment Time *' with a green clock icon and the value '16:00', and 'Contact Phone *' with a green phone icon and the value '9632586853'.

Figure C.6: Doctor Appointment Schedule

← Edit Medicine ?



Edit Medicine Details
Update the medicine information as needed

Medicine ID: 10

 **Update Medicine Information**

Medicine Name *
 Fibroadenosis

Medicine Type *
 Tablet ▼

Dosage Strength *
 500mg

Stock Quantity *
 10

Description *
Reducing caffeine and

Figure C.7: Medicine Details

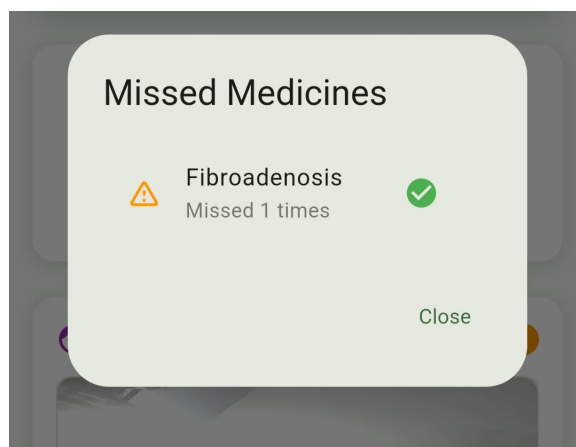


Figure C.8: Missed Medicine

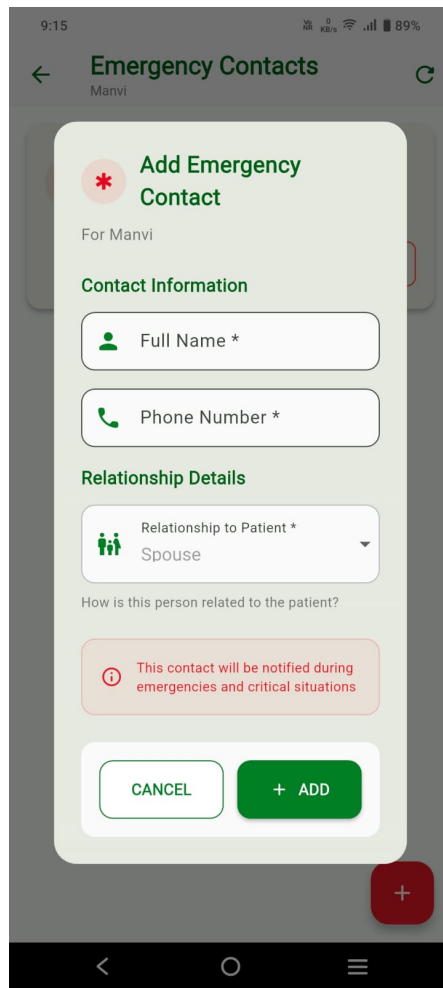


Figure C.9: Emergency Contact

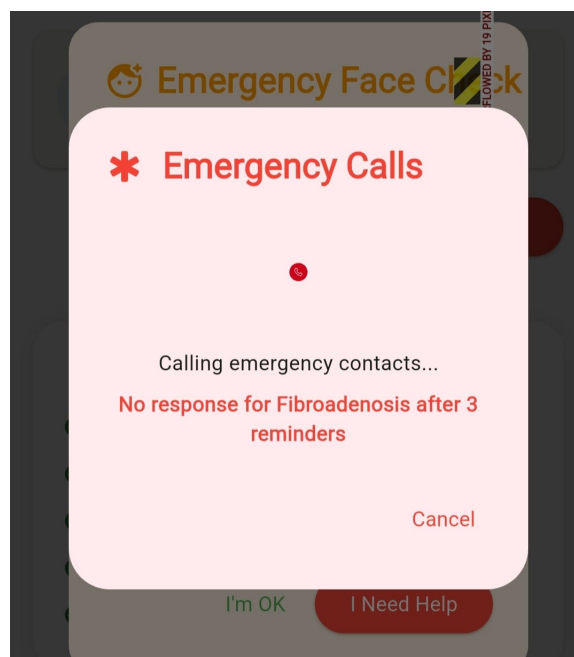


Figure C.10: Emergency Calls

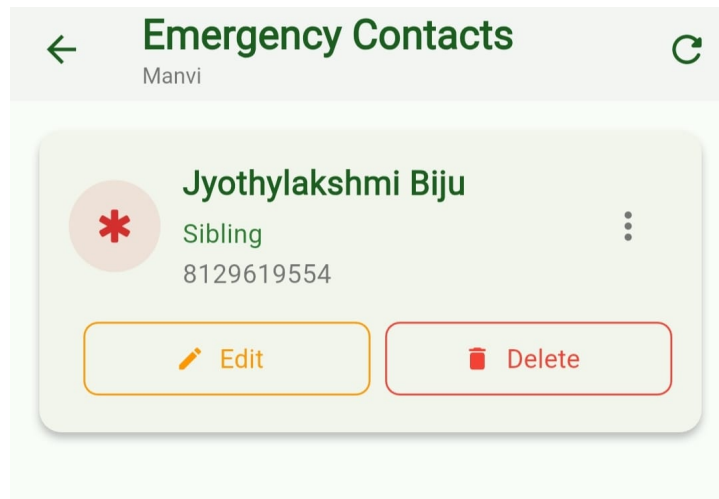


Figure C.11: Contact Person

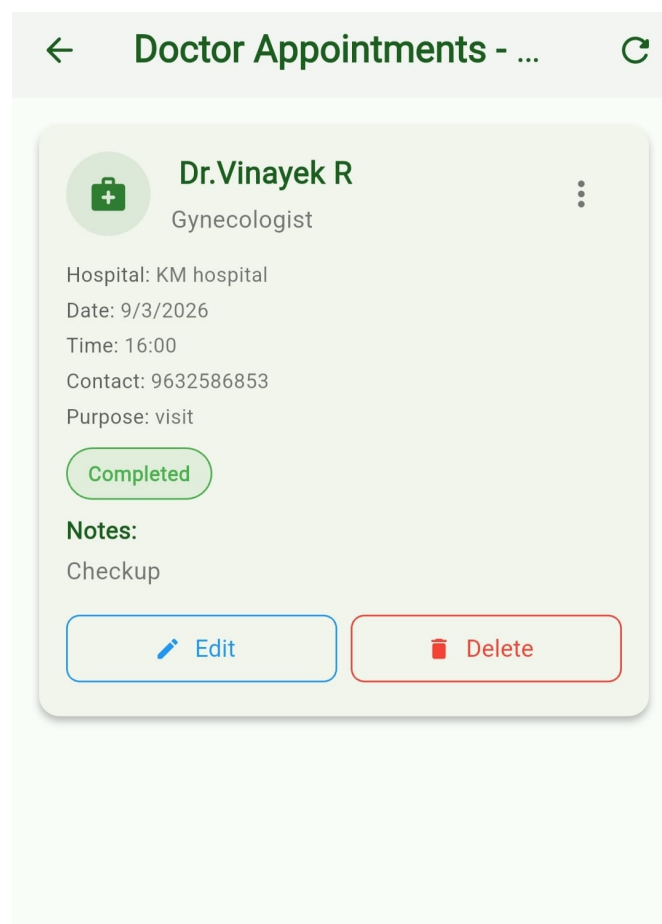


Figure C.12: Doctor Appointments

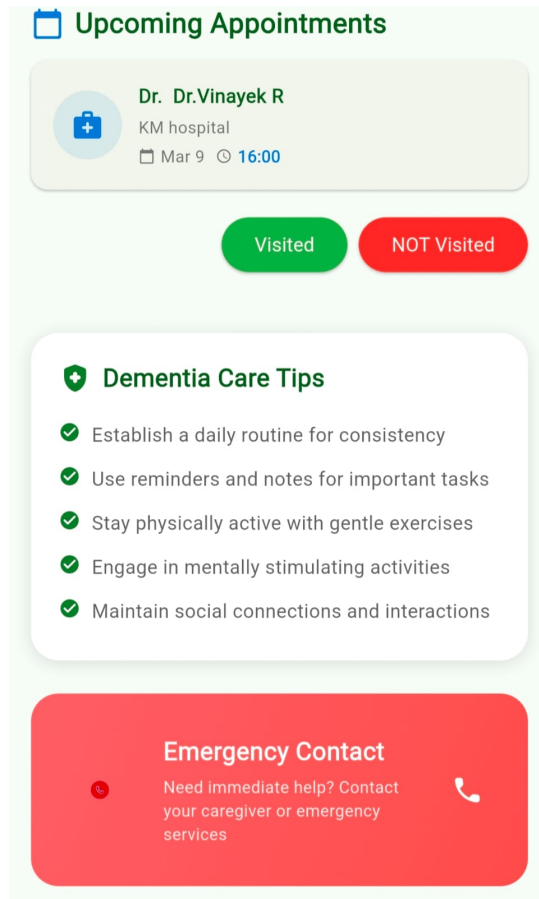


Figure C.13: Upcoming Appointments

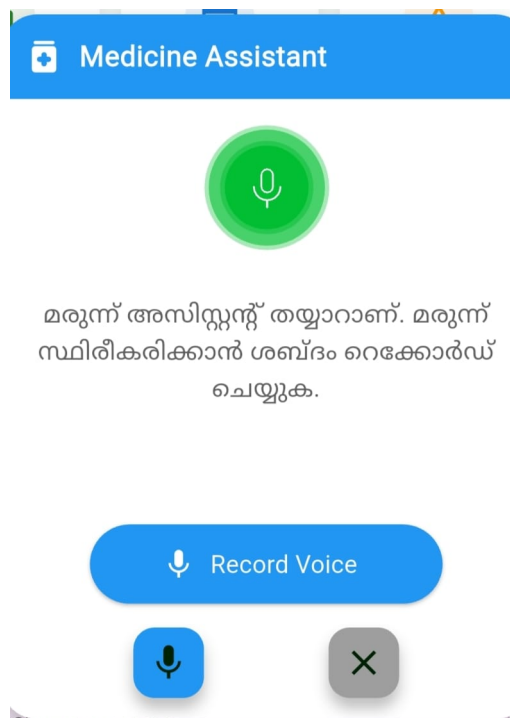


Figure C.14: Voice Command

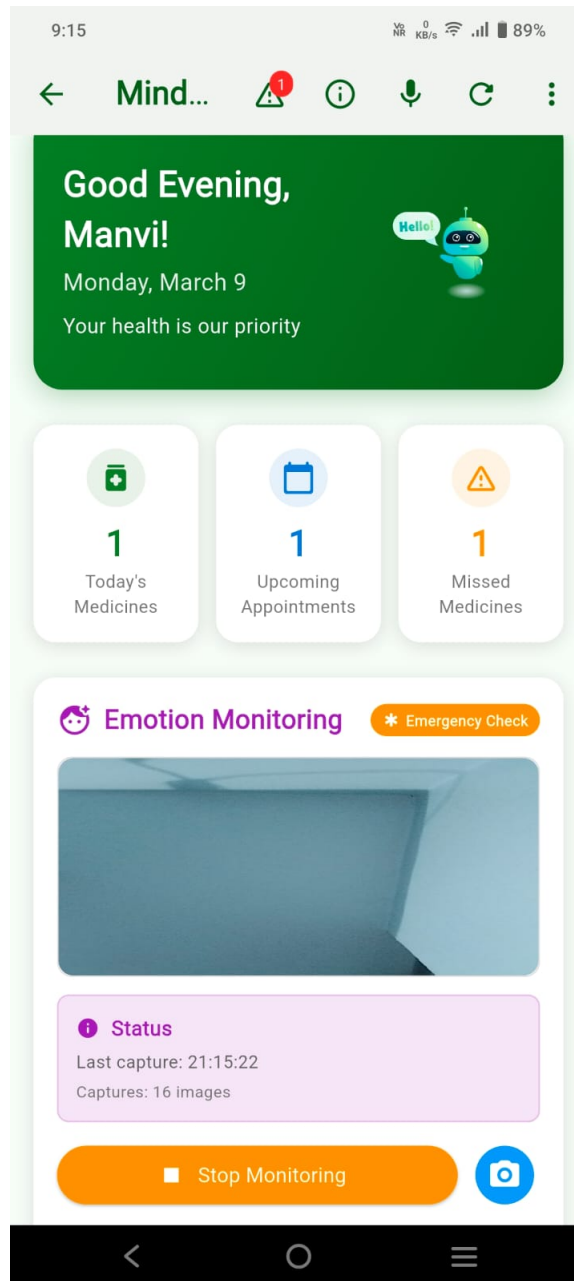


Figure C.15: Patient's Home Page